

Genre Coherence in Spotify’s Audio Feature Space

STAT 4710/5710 — Modern Data Mining, Spring 2026

Sung Cho, Etienne Lee, Leo Li

2026-04-27

Contents

Executive Summary	1
Introduction and Motivation	1
Data Description	2
Exploratory Data Analysis	2
Methods	7
Results	10
Discussion and Conclusion	38
Statement on AI Use	39
Appendix	40

Executive Summary

Music streaming platforms assign genre labels to tracks as a primary means of organization and discovery, yet it remains unclear whether these labels reflect genuine musical coherence or function primarily as marketing categories. This project investigates the relationship between Spotify’s genre taxonomy and the underlying audio feature space using a dataset of 114,000 tracks spanning 125 genres. We apply principal component analysis to reduce the high-dimensional audio feature space and use k-means and hierarchical clustering to assess whether unsupervised groupings recover genre labels, quantified by adjusted Rand index and silhouette score. We further compute within-genre variance across all 125 genres to produce a genre coherence ranking, train a multi-class classifier to identify boundary songs that are musically proximate to genres other than their assigned label, and fit lasso regression models — both globally and stratified by macro-genre — to assess whether genre label predicts track popularity beyond what audio features alone explain. Together, these analyses ask whether genre is a meaningful descriptor of musical identity or an artifact of industry convention, with implications for how streaming platforms structure discovery and recommendation.

Introduction and Motivation

Genre is one of the most fundamental organizing principles in music. Streaming platforms like Spotify use genre labels to drive recommendation algorithms, playlist curation, and discovery features. Yet the boundary between genres has always been porous: artists blend styles, listeners resist categorization, and industry labels often reflect marketing logic rather than musical content. This project uses Spotify’s audio feature API — a set of continuous measurements derived from signal processing — to ask a concrete question: do songs within the same genre actually sound similar to each other?

We approach this question from five angles: unsupervised clustering (does the audio feature space cluster into genre-like groups?), within-genre variance (which genres are musically tight vs. diffuse?), popularity

regression (does genre predict popularity beyond what audio features explain?), boundary song detection (which songs are misclassified by a genre classifier, and what does that reveal?), and feature importance (which audio dimensions drive genre separation vs. popularity?). Together these form a comprehensive audit of Spotify’s genre taxonomy.

Data Description

The dataset is the Spotify Tracks Dataset from Kaggle (author: maharshipandya), comprising approximately 114,000 tracks across 125 fine-grained genres. Each row corresponds to a unique track with 20 columns: metadata (track ID, artist, album, name), one outcome variable (**popularity**, scored 0–100 based on recent play counts), three discrete audio features (**key**, **mode**, **time_signature**), and ten continuous audio features that form our primary feature matrix.

The ten continuous features used throughout this analysis are: **danceability**, **energy**, **loudness**, **speechiness**, **acousticness**, **instrumentalness**, **liveness**, **valence**, **tempo**, and **duration_ms**. The discrete features (**key**, **mode**, **time_signature**) are excluded from the primary feature matrix but are explored in EDA. After removing duplicate track IDs and rows with missing values in the audio features, the cleaned dataset contains 89741 tracks across 113 genres.

We map the 125 fine-grained genres to 11 macro-genre families (e.g., Pop, Rock/Metal, Electronic/Dance) to enable higher-level comparisons and visualizations. Any genres not covered by the mapping are labeled “Other.”

```
tibble(
  Statistic = c("Total tracks (after dedup & NA removal)",
               "Unique fine-grained genres",
               "Unique macro-genres",
               "Features in primary matrix"),
  Value = c(nrow(spotify),
            n_distinct(spotify$track_genre),
            n_distinct(spotify$macro_genre),
            length(audio_features))
) %>%
kable(caption = "Dataset summary after preprocessing") %>%
kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)
```

Table 1: Dataset summary after preprocessing

Statistic	Value
Total tracks (after dedup & NA removal)	89741
Unique fine-grained genres	113
Unique macro-genres	11
Features in primary matrix	10

Exploratory Data Analysis

We begin with univariate and bivariate summaries of the audio features and genre distribution to understand the data structure before modeling.

Distributions of Audio Features and Popularity

```
# Popularity histogram
p_pop <- ggplot(spotify, aes(x = popularity)) +
  geom_histogram(bins = 50, fill = "#3498db", color = "white", alpha = 0.8) +
  labs(title = "Distribution of Popularity", x = "Popularity (0-100)", y = "Count") +
  theme_minimal()

# Audio feature density plots
spotify_long <- spotify %>%
  select(all_of(audio_features)) %>%
  pivot_longer(everything(), names_to = "feature", values_to = "value")

p_features <- ggplot(spotify_long, aes(x = value)) +
  geom_histogram(bins = 40, fill = "#2ecc71", color = "white", alpha = 0.8) +
  facet_wrap(~feature, scales = "free", ncol = 5) +
  labs(title = "Distributions of Audio Features", x = NULL, y = "Count") +
  theme_minimal() +
  theme(strip.text = element_text(size = 9))

print(p_pop)
```

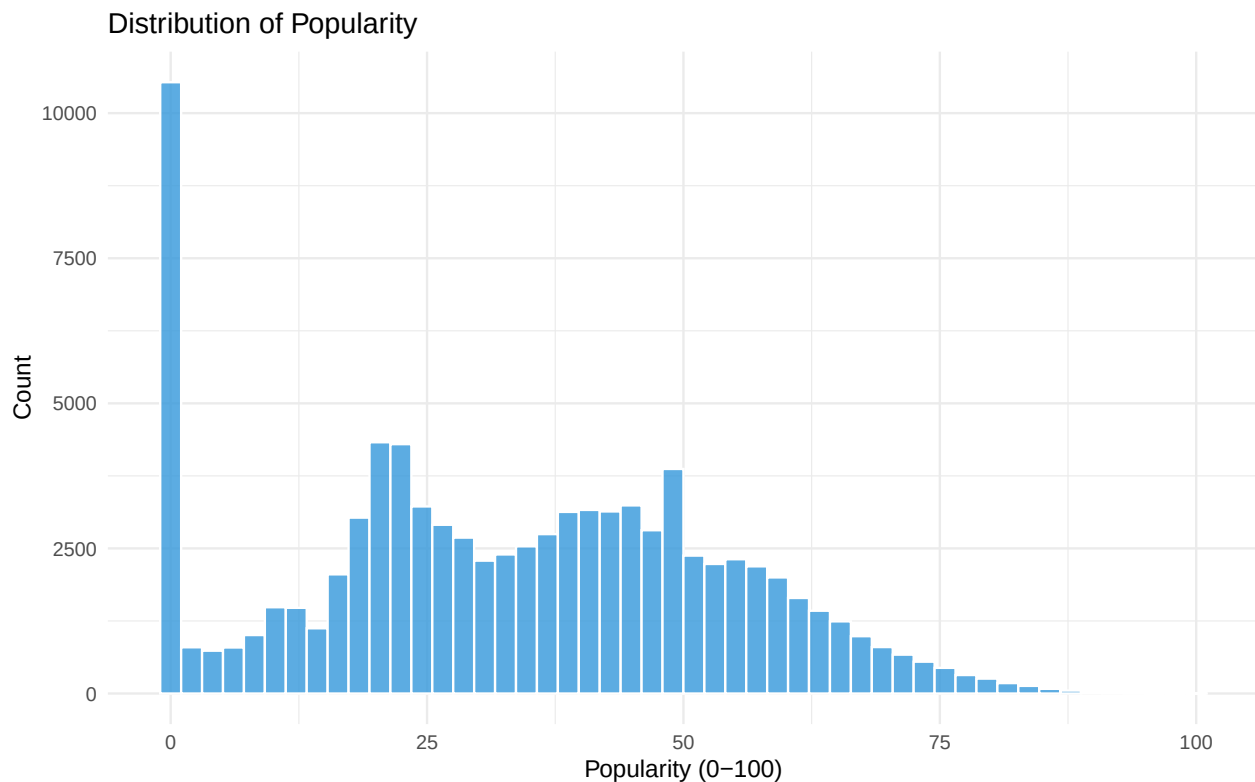


Figure 1: Distributions of popularity and the 10 audio features. Note the spike at popularity = 0 and the strong bimodality of instrumentality.

```
print(p_features)
```

The popularity distribution is right-skewed with a pronounced spike at 0, reflecting the large number of obscure tracks. instrumentality is highly bimodal: most tracks are either strongly vocal or strongly

Distributions of Audio Features

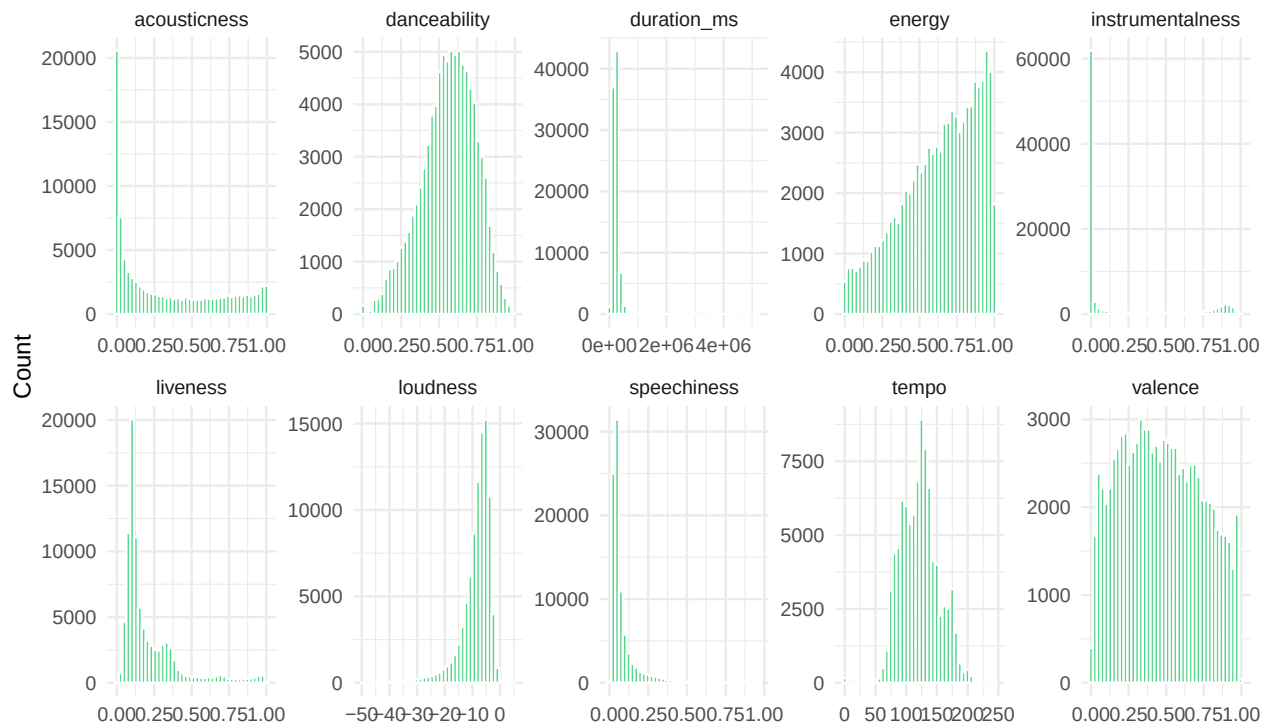


Figure 2: Distributions of popularity and the 10 audio features. Note the spike at popularity = 0 and the strong bimodality of instrumentalness.

instrumental with few intermediate values. `speechiness` and `acousticness` are also right-skewed.

Correlation Structure

```
cor_mat <- cor(spotify %>% select(all_of(audio_features)), use = "complete.obs")

corrplot(cor_mat,
  method = "color",
  type = "upper",
  order = "hclust",
  tl.cex = 0.85,
  addCoef.col = "black",
  number.cex = 0.65,
  col = colorRampPalette(c("#e74c3c", "white", "#2980b9"))(200),
  title = "Audio Feature Correlation Matrix",
  mar = c(0, 0, 1, 0))
```

As expected, `energy` and `loudness` are strongly positively correlated (~ 0.76), while `energy` and `acousticness` are strongly negatively correlated (~ -0.71). `valence` and `danceability` show moderate positive correlation (~ 0.46). These relationships motivate dimensionality reduction via PCA.

Genre Counts by Macro-Genre

```
spotify %>%
  group_by(macro_genre) %>%
```

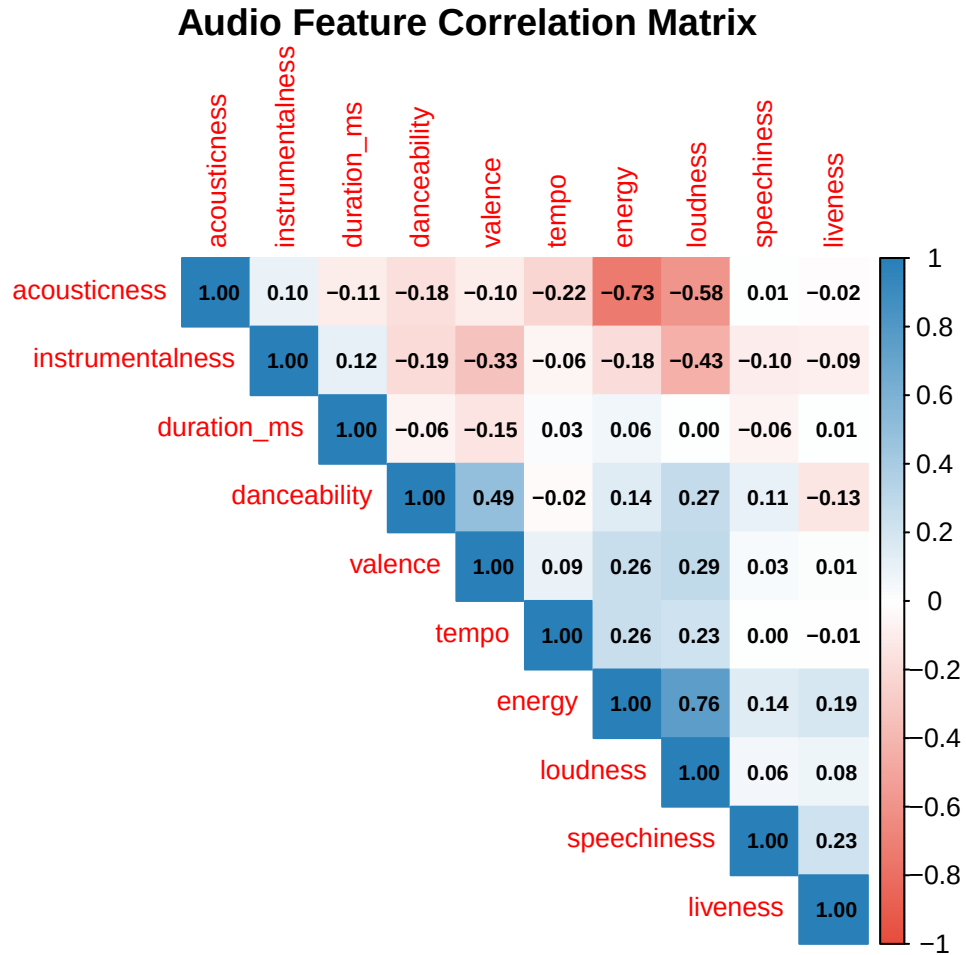


Figure 3: Correlation matrix of the 10 audio features. Strong positive correlation between energy and loudness; strong negative correlation between energy and acousticalness.

```

summarise(n = n(), .groups = "drop") %>%
  arrange(desc(n)) %>%
  ggplot(aes(x = reorder(macro_genre, n), y = n, fill = macro_genre)) +
  geom_col(show.legend = FALSE) +
  coord_flip() +
  scale_fill_brewer(palette = "Set3") +
  labs(title = "Track Counts by Macro-Genre",
       x = "Macro-Genre", y = "Number of Tracks") +
  theme_minimal()

```

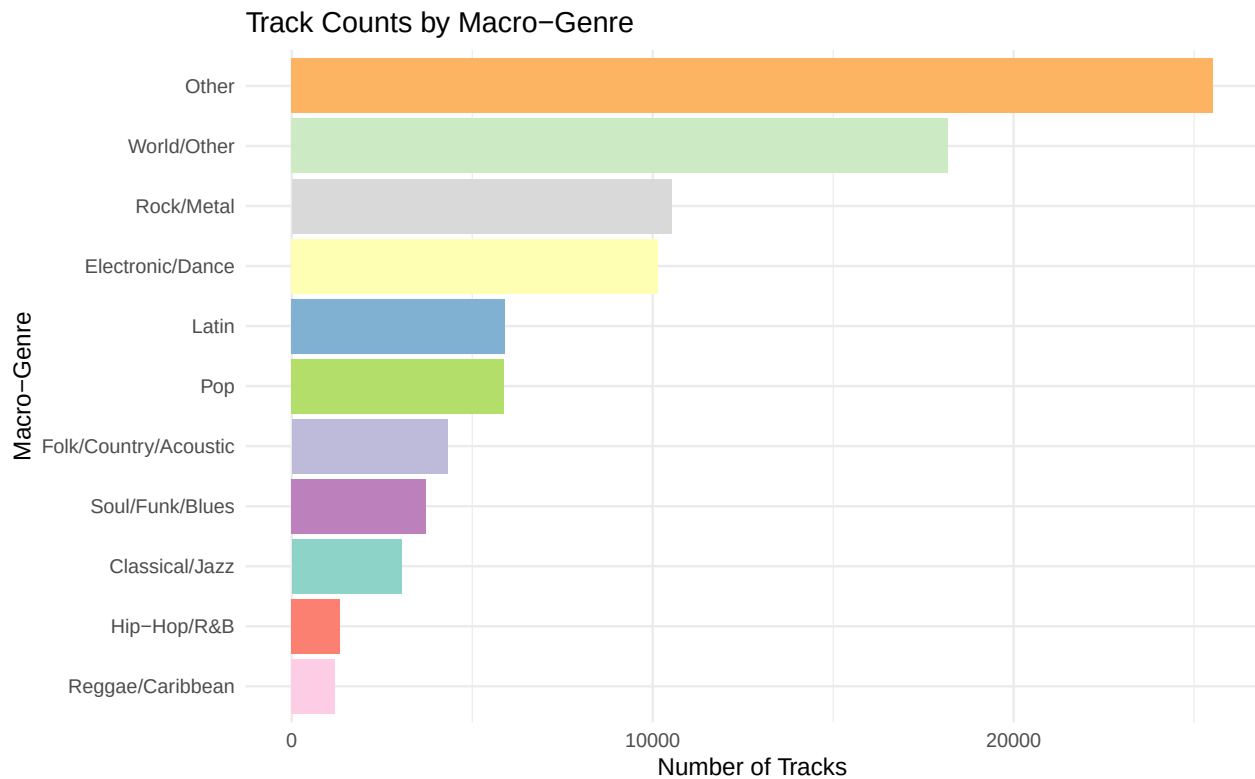


Figure 4: Track counts by macro-genre. Some macro-genre families are substantially larger than others, reflecting the original dataset composition.

Audio Fingerprint by Macro-Genre (Radar Chart)

```

# Compute mean standardized features per macro-genre
radar_data <- as_tibble(X_scaled) %>%
  mutate(macro_genre = spotify$macro_genre) %>%
  group_by(macro_genre) %>%
  summarise(across(everything(), mean), .groups = "drop")

genres_list <- levels(spotify$macro_genre)

# Radar chart requires max/min rows prepended
radar_mat <- radar_data %>%
  select(-macro_genre) %>%
  as.data.frame()
rownames(radar_mat) <- radar_data$macro_genre

```

```

max_vals <- apply(radar_mat, 2, max) + 0.1
min_vals <- apply(radar_mat, 2, min) - 0.1
radar_plot_data <- rbind(max_vals, min_vals, radar_mat)

par(mfrow = c(3, 4), mar = c(1, 1, 2, 1))
colors_radar <- RColorBrewer::brewer.pal(min(nrow(radar_mat), 11), "Set3")
for (i in seq_len(nrow(radar_mat))) {
  fmsb::radarchart(
    radar_plot_data[c(1, 2, i + 2), ],
    axistype = 1,
    pcol = colors_radar[(i - 1) %% length(colors_radar) + 1],
    pfcol = adjustcolor(colors_radar[(i - 1) %% length(colors_radar) + 1], alpha.f = 0.3),
    plwd = 2,
    cglcol = "grey80",
    cglty = 1,
    axislabcol = "grey50",
    vlce = 0.65,
    title = rownames(radar_mat)[i]
  )
}
par(mfrow = c(1, 1))

```

Popularity by Macro-Genre

```

ggplot(spotify, aes(x = reorder(macro_genre, popularity, median),
                        y = popularity, fill = macro_genre)) +
  geom_boxplot(outlier.size = 0.3, outlier.alpha = 0.3, show.legend = FALSE) +
  coord_flip() +
  scale_fill_brewer(palette = "Set3") +
  labs(title = "Popularity by Macro-Genre",
       x = "Macro-Genre", y = "Popularity (0-100)") +
  theme_minimal()

```

Popularity differs meaningfully across macro-genres, with Hip-Hop/R&B showing the highest median popularity at roughly 47, followed by Pop at about 41, while Latin, World/Other, and Electronic/Dance cluster in the middle around the mid-30s to high-30s. At the lower end, Classical/Jazz has the lowest median, near 20, and Other is also relatively low at around 27. In terms of spread, several genres show substantial variability: Hip-Hop/R&B, Pop, and Electronic/Dance have wide interquartile ranges and long upper whiskers, suggesting these categories contain both many low-visibility songs and a meaningful number of breakout tracks. Latin and Other also display notable high-end outliers, with some songs reaching the 80–95 range despite having lower central tendency overall. By contrast, Classical/Jazz is concentrated at lower popularity levels and appears less likely to produce extremely popular tracks. Qualitatively, this pattern matches streaming behavior: mainstream, playlist-driven genres such as Hip-Hop/R&B and Pop tend to dominate mass listening and algorithmic discovery, while genres like Classical/Jazz or Folk/Country/Acoustic may have more niche but stable audiences, leading to lower medians even when a few songs still achieve broad reach.

Methods

This section describes the five analytical angles applied to the dataset. All methods operate on the standardized 10-feature matrix unless otherwise noted.



Figure 5: Radar charts showing the mean standardized audio profile ('fingerprint') for each macro-genre family.

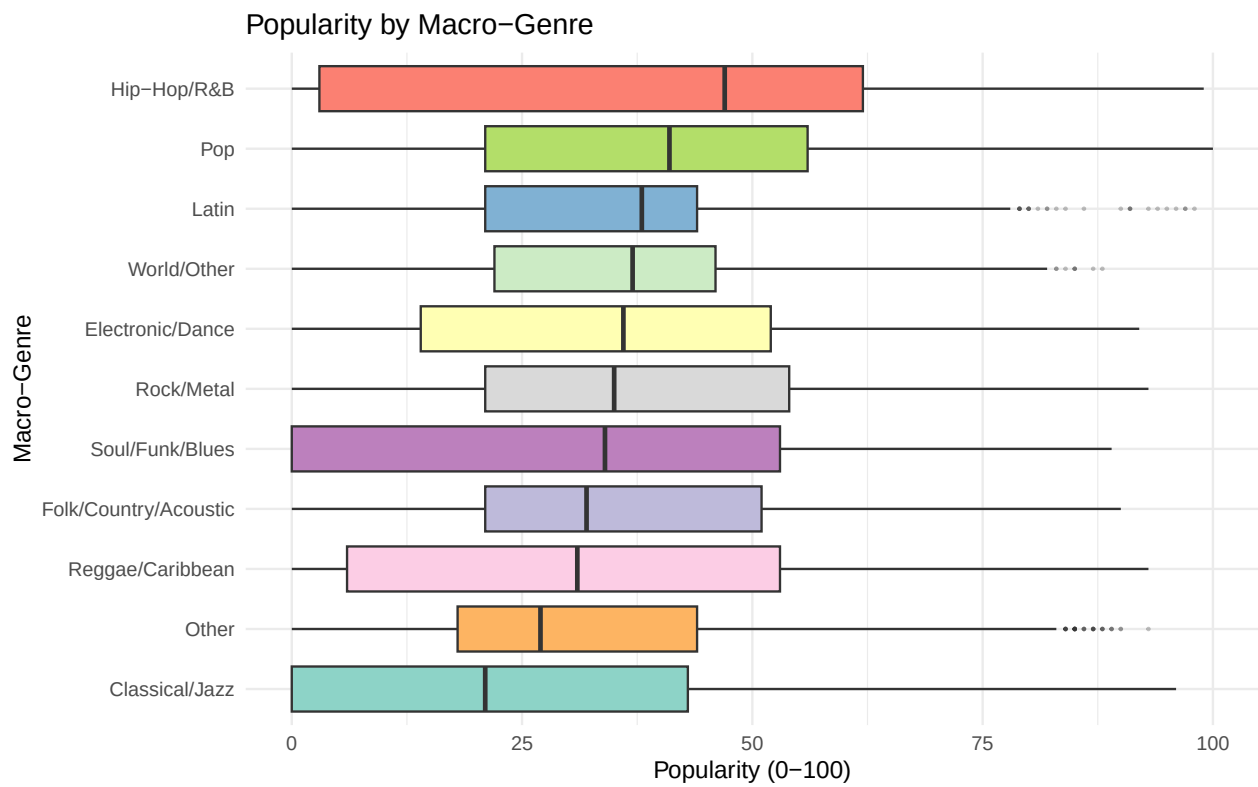


Figure 6: Distribution of popularity scores by macro-genre. Pop and Hip-Hop/R&B tend to have higher median popularity; Classical/Jazz and Electronic/Dance show wider variance.

PCA and Clustering

We use PCA to project the 10-dimensional audio feature space into a lower-dimensional space for visualization and as input to clustering. K-means and hierarchical clustering are then applied to assess whether unsupervised groupings align with genre labels.

Within-Genre Variance

For each of the 125 fine-grained genres, we compute the mean within-genre variance across all standardized features, yielding a coherence score. Genres with low variance (high coherence) have tracks that sound similar to each other; diffuse genres have high internal variance.

Popularity Regression

We model `popularity` as a function of audio features and macro-genre dummies using OLS, lasso (global), and stratified OLS per macro-genre. This tests whether genre labels carry market value beyond what audio features explain.

Boundary Song Detection

A random forest classifier is trained to predict macro-genre from audio features. Tracks where the true genre receives low predicted probability — while another genre receives the highest probability — are termed “boundary songs” and reveal where genre assignments are musically ambiguous.

Feature Importance

We synthesize classifier variable importance (random forest) with lasso coefficient magnitudes to ask which audio dimensions drive genre separation vs. popularity prediction.

Audio Feature Geometry

Inspired by word2vec-style vector arithmetic, we define interpretable axes in the 10-dimensional audio feature space by computing normalized direction vectors between pairs of maximally distinct genre centroids. Projecting all genre centroids onto these axes tests whether the space has semantic directionality — genres sorting sensibly along acoustic, vocal, and intensity dimensions — even in the absence of clean cluster separation.

Results

Angle 1: PCA and Clustering

```
set.seed(42)

pca_fit <- prcomp(X_scaled, scale. = FALSE) # already standardized

# Scree plot
fviz_screplot(pca_fit, addlabels = TRUE, ncp = 10,
              barfill = "#3498db", barcolor = "white",
              linecolor = "#e74c3c") +
  labs(title = "PCA Scree Plot - Variance Explained per Component") +
  theme_minimal()
```

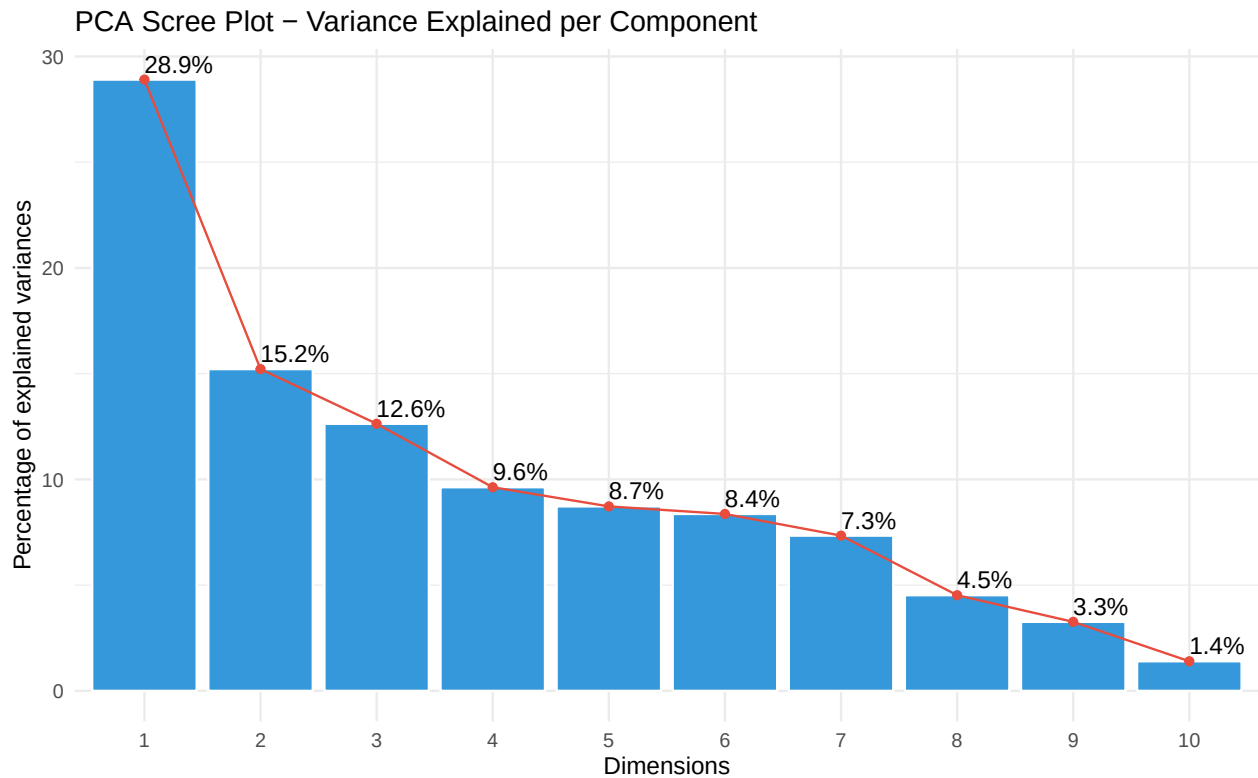


Figure 7: Left: Scree plot showing variance explained by each principal component. Right: PCA biplot of PC1 vs. PC2 colored by macro-genre.

```
# Biplot PC1 vs PC2
pca_scores <- as_tibble(pca_fit$x) %>%
  mutate(macro_genre = spotify$macro_genre)

ggplot(pca_scores %>% sample_n(5000), aes(x = PC1, y = PC2, color = macro_genre)) +
  geom_point(alpha = 0.4, size = 0.8) +
  scale_color_brewer(palette = "Set3") +
  labs(title = "PCA Biplot: PC1 vs PC2 (random sample of 5,000 tracks)",
       color = "Macro-Genre") +
  theme_minimal() +
  guides(color = guide_legend(override.aes = list(size = 3, alpha = 1)))

# Cumulative variance explained
cum_var <- cumsum(pca_fit$sdev^2) / sum(pca_fit$sdev^2)
tibble(
  Components = c(2, 3, 5, 10),
  `Cumulative Variance Explained` = scales::percent(cum_var[c(2, 3, 5, 10)])
) %>%
  kable(caption = "Cumulative variance explained by PCA components") %>%
  kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)
```

PCA

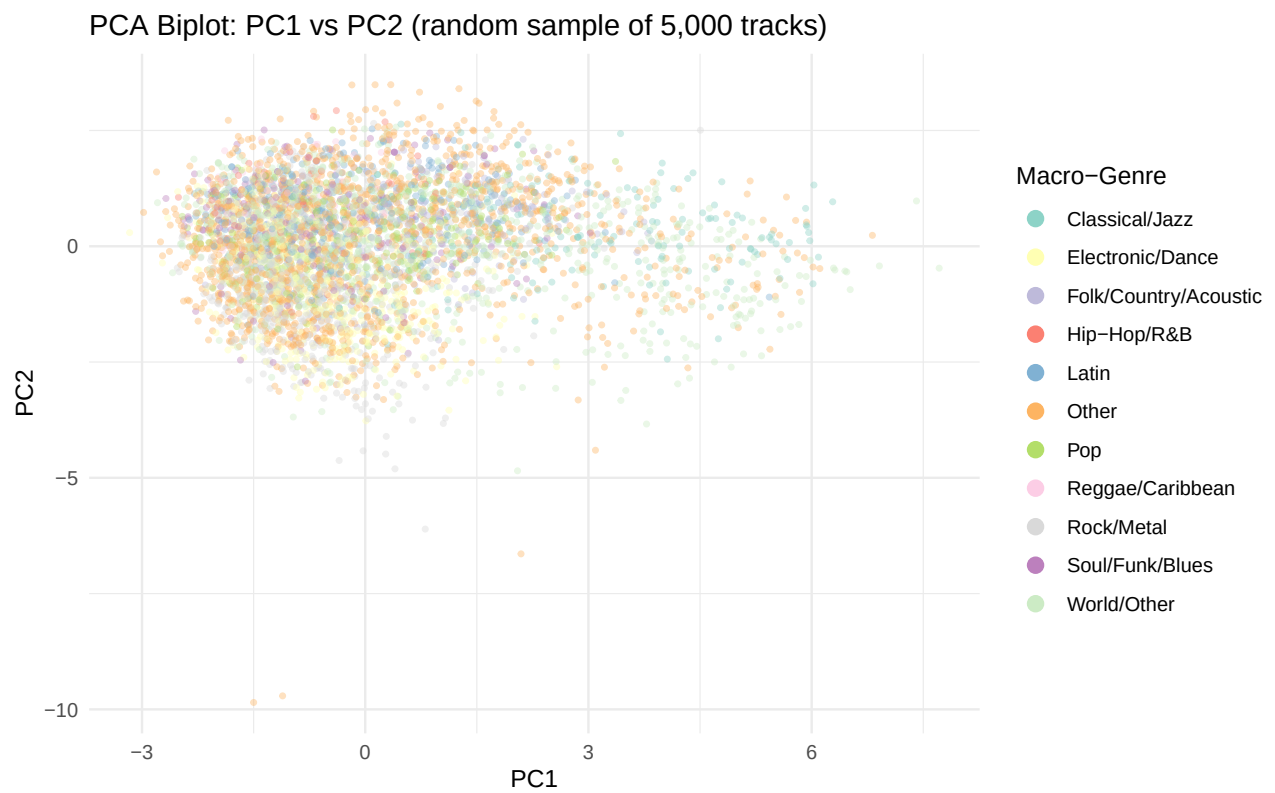


Figure 8: Left: Scree plot showing variance explained by each principal component. Right: PCA biplot of PC1 vs. PC2 colored by macro-genre.

Table 2: Cumulative variance explained by PCA components

Components	Cumulative Variance Explained
2	44%
3	57%
5	75%
10	100%

The PCA results show that the first few components capture a substantial share of the variation in Spotify audio features, but not enough to reduce the genre structure to just one or two clean dimensions. Quantitatively, PC1 explains 28.9% of the total variance and PC2 explains 15.2%, so the first two components together account for about 44% of the variation. Expanding further, the first 3 PCs explain 57%, and the first 5 PCs explain 75%, which suggests that the dataset has meaningful lower-dimensional structure, but that this structure is still moderately high-dimensional rather than dominated by a single latent axis. The scree plot shows a clear drop after the first component and then a more gradual decline, indicating that PC1 is especially important, while PCs 2–5 still each contribute nontrivially. Qualitatively, however, the PC1-versus-PC2 biplot shows substantial overlap across macro-genres, with no sharply separated clusters. Some genres appear to drift toward different regions—for example, World/Other and Classical/Jazz extend further to the right on PC1, while several mainstream genres such as Pop, Hip-Hop/R&B, Latin, and Electronic/Dance concentrate more heavily near the center-left—but the boundaries remain highly diffuse. Overall, the PCA suggests that while audio features do contain structured variation, the first two principal components do not visually separate macro-genres cleanly, which is early evidence that Spotify genre labels may overlap considerably in underlying musical feature space.

UMAP Projection Unlike PCA, which finds the linear directions of maximum variance, UMAP (Uniform Manifold Approximation and Projection) preserves local neighborhood structure in a non-linear 2D embedding. It often reveals cluster separation that PCA cannot show.

```
set.seed(42)

idx_umap <- sample(nrow(X_scaled), min(10000, nrow(X_scaled)))
X_umap_in <- X_scaled[idx_umap, ]
macro_umap <- spotify$macro_genre[idx_umap]

umap_coords <- uwot::umap(X_umap_in, n_neighbors = 15, min_dist = 0.1,
                          n_components = 2, verbose = FALSE)

umap_df <- tibble(
  UMAP1 = umap_coords[, 1],
  UMAP2 = umap_coords[, 2],
  macro_genre = macro_umap
)

ggplot(umap_df, aes(x = UMAP1, y = UMAP2, color = macro_genre)) +
  geom_point(alpha = 0.35, size = 0.7) +
  scale_color_brewer(palette = "Set3") +
  labs(title = "UMAP Projection of Audio Feature Space (10,000 tracks)",
       color = "Macro-Genre") +
  theme_minimal() +
  guides(color = guide_legend(override.aes = list(size = 3, alpha = 1)))
```

The UMAP projection reveals a strikingly clear result: nearly all macro-genres collapse into a single large, undifferentiated mass. Pop, Rock/Metal, Hip-Hop/R&B, Latin, Folk/Country/Acoustic, Soul/Funk/Blues,

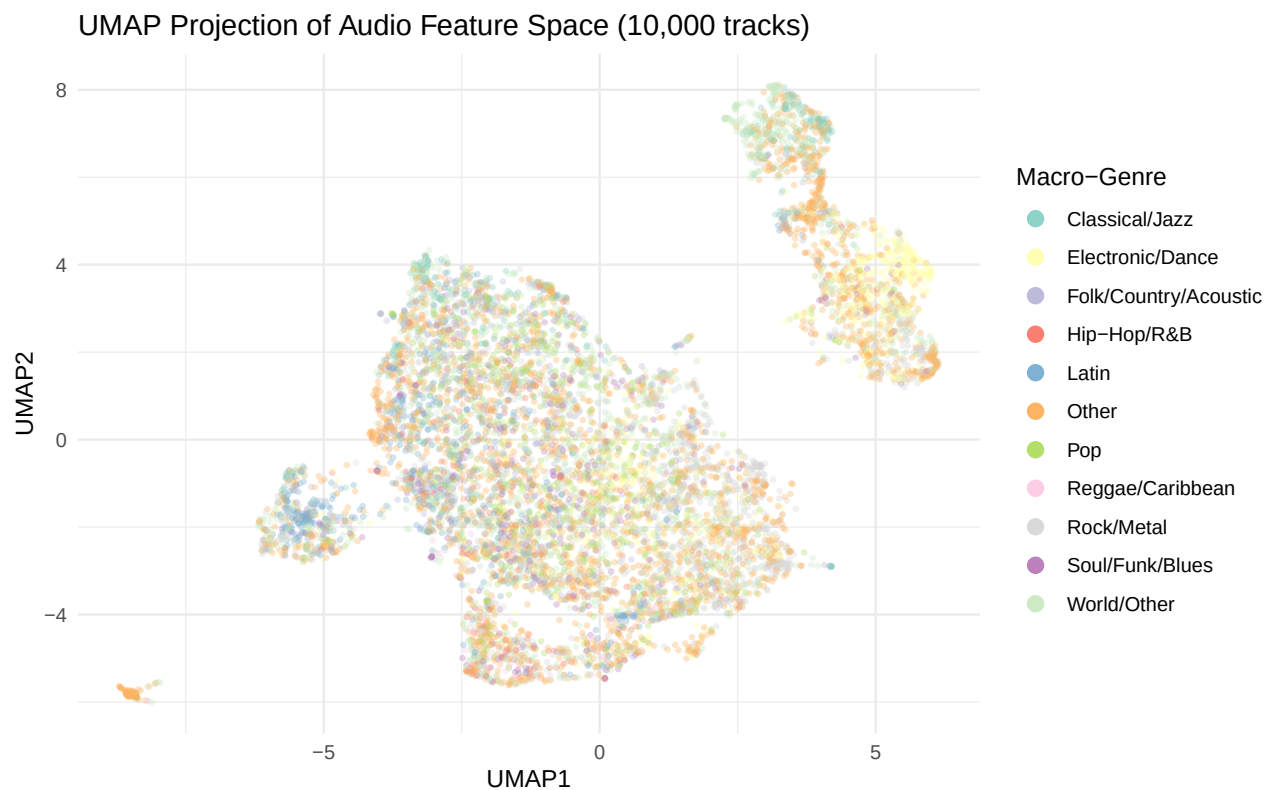


Figure 9: UMAP 2D projection of the standardized audio feature space (10,000 tracks), colored by macro-genre. Compared to the PCA biplot, UMAP better preserves local neighborhood structure and can reveal non-linear separation between genre families.

and World/Other are thoroughly intermingled throughout the central blob, with no discernible boundaries between them even under UMAP's non-linear lens.

The one meaningful exception is Classical/Jazz, which forms a partially separated cluster in the upper-right region of the embedding. This aligns with its known audio fingerprint — distinctively high instrumentalness and acousticness, low energy and danceability — making it the only macro-genre that is genuinely acoustically coherent and spatially separable. A small isolated cluster also appears at the far bottom-left, consisting of tracks with unusual feature combinations far from the genre mainstream.

Crucially, this null result is more informative than it might appear. UMAP is the most powerful tool available for revealing hidden cluster structure — unlike PCA, it is not constrained to linear projections and will surface any non-linear geometry in the data. The fact that genres remain inseparable even under UMAP closes the strongest counterargument to the low ARI finding from clustering: there is no non-linear structure that PCA was simply missing. Genre boundaries are genuinely not encoded in these 10 audio features, with the single exception of Classical/Jazz.

```
set.seed(42)

k_values <- c(5, 10, 15, 20, 30)
wcss <- numeric(length(k_values))
sil_scores <- numeric(length(k_values))

# Subsample for speed (hierarchical will use same subsample)
set.seed(42)
idx_sub <- sample(nrow(X_scaled), min(10000, nrow(X_scaled)))
X_sub <- X_scaled[idx_sub, ]

for (i in seq_along(k_values)) {
  km <- kmeans(X_sub, centers = k_values[i], nstart = 10, iter.max = 50)
  wcss[i] <- km$tot.withinss
  sil <- silhouette(km$cluster, dist(X_sub))
  sil_scores[i] <- mean(sil[, 3])
}

elbow_df <- tibble(k = k_values, wcss = wcss, silhouette = sil_scores)

p_elbow <- ggplot(elbow_df, aes(x = k, y = wcss)) +
  geom_line(color = "#3498db", linewidth = 1) +
  geom_point(size = 3, color = "#2980b9") +
  labs(title = "Elbow Plot: WCSS vs. k", x = "k", y = "Within-Cluster SS") +
  theme_minimal()

p_sil <- ggplot(elbow_df, aes(x = k, y = silhouette)) +
  geom_line(color = "#e74c3c", linewidth = 1) +
  geom_point(size = 3, color = "#c0392b") +
  labs(title = "Silhouette Score vs. k", x = "k", y = "Mean Silhouette") +
  theme_minimal()

gridExtra::grid.arrange(p_elbow, p_sil, ncol = 2)

# Select best k
best_k <- k_values[which.max(sil_scores)]
cat("Selected k:", best_k, "\n")
```

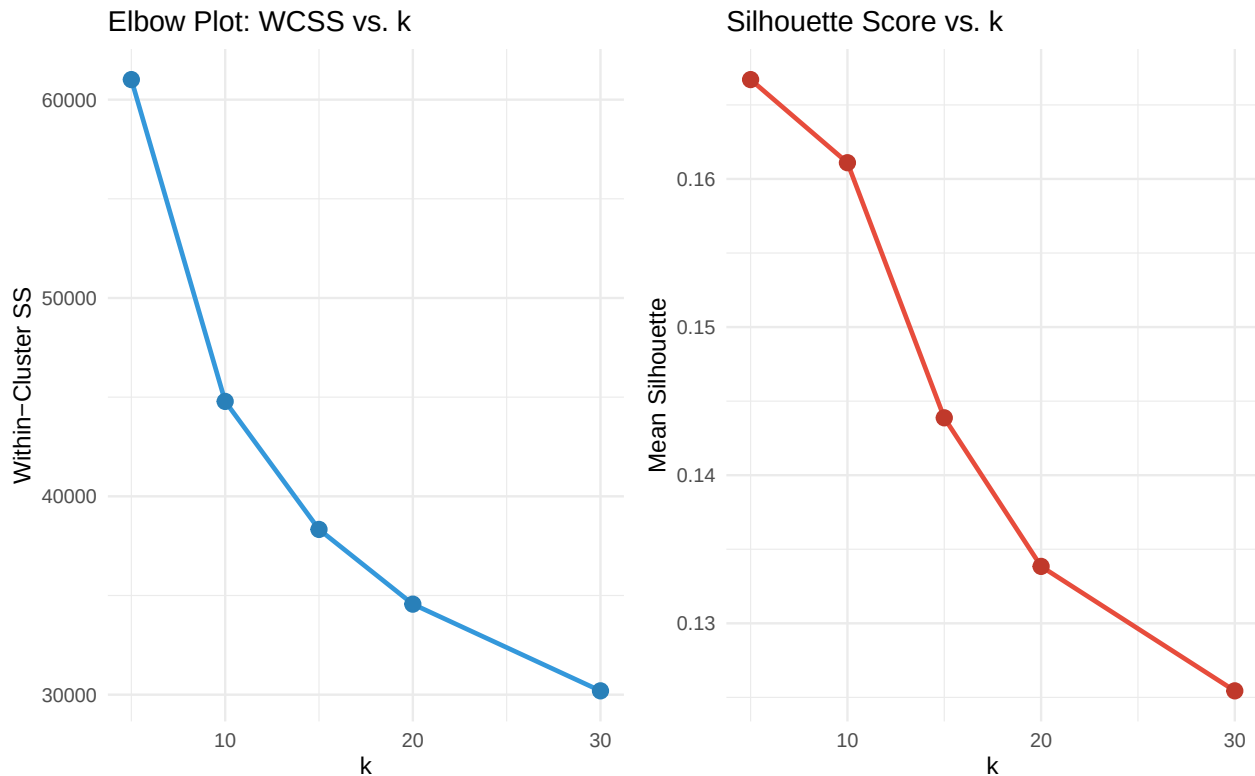


Figure 10: Elbow plot (WCSS) and silhouette scores for k-means across $k = 5$ to 30. The selected k balances the elbow heuristic and peak silhouette.

K-Means Clustering

Selected k: 5

```
# Final k-means at best k
set.seed(42)
km_best <- kmeans(X_sub, centers = best_k, nstart = 20, iter.max = 100)

# ARI against macro_genre
macro_sub <- spotify$macro_genre[idx_sub]
ari_kmeans <- adjustedRandIndex(km_best$cluster, as.integer(macro_sub))
cat(sprintf("K-means ARI (k=%d) vs. macro_genre: %.4f\n", best_k, ari_kmeans))
```

K-means ARI (k=5) vs. macro_genre: 0.0210

```
# Visualize on PC1-PC2
pca_sub <- as_tibble(pca_fit$x[idx_sub, 1:2]) %>%
  mutate(cluster = factor(km_best$cluster),
         macro_genre = macro_sub)

ggplot(pca_sub, aes(x = PC1, y = PC2, color = cluster)) +
  geom_point(alpha = 0.3, size = 0.7) +
  labs(title = sprintf("K-Means Clusters (k=%d) on PC1-PC2", best_k),
       color = "Cluster") +
  theme_minimal()
```

The k-means results provide strong quantitative evidence that unsupervised audio-feature clusters do not

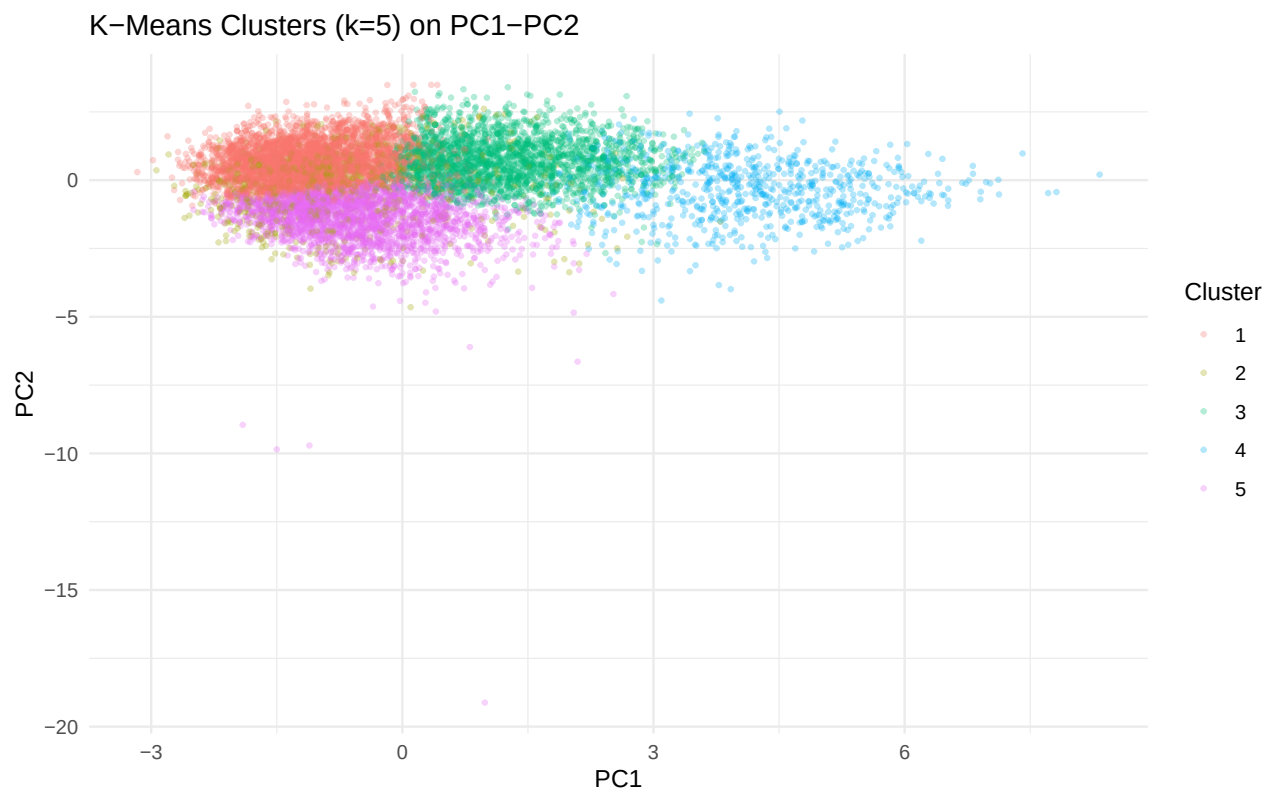


Figure 11: Elbow plot (WCSS) and silhouette scores for k-means across $k = 5$ to 30. The selected k balances the elbow heuristic and peak silhouette.

align closely with Spotify’s macro-genre labels. Using the elbow plot and silhouette scores, the selected solution was $k=5$: WCSS falls sharply at first and then begins to flatten, while the mean silhouette score is highest at the smallest tested value and declines steadily as k increases, suggesting that adding more clusters does not create cleaner separation. However, when these five clusters are compared to the observed macro-genres, the adjusted Rand index is only 0.0210, which is extremely close to 0. Since an ARI near 0 indicates near-random agreement and an ARI near 1 indicates perfect recovery, this value implies that the cluster assignments recover almost none of the labeled genre structure beyond chance. Visually, the PC1–PC2 cluster plot confirms this interpretation: the algorithm finds geometric groupings in feature space, but these groupings look like broad overlapping clouds rather than clear genre-specific partitions. In other words, while songs do organize into clusters based on audio similarity, those clusters do not map meaningfully onto Spotify’s macro-genre taxonomy. This low ARI is therefore not a side result but one of the central findings of the project, suggesting that genre labels on Spotify reflect a much noisier and more culturally constructed categorization than a clean partition of songs by underlying acoustic characteristics.

```
set.seed(42)

# Use smaller subsample for hierarchical (distance matrix is  $O(n^2)$ )
idx_hier <- sample(nrow(X_scaled), min(3000, nrow(X_scaled)))
X_hier <- X_scaled[idx_hier, ]
macro_hier <- spotify$macro_genre[idx_hier]

dist_mat <- dist(X_hier)
hc_fit <- hclust(dist_mat, method = "ward.D2")

hc_cut <- cutree(hc_fit, k = best_k)
ari_hclust <- adjustedRandIndex(hc_cut, as.integer(macro_hier))

tibble(
  Method = c("K-Means", "Hierarchical (Ward)"),
  k = best_k,
  ARI = round(c(ari_kmeans, ari_hclust), 4)
) %>%
  kable(caption = "Adjusted Rand Index: clustering vs. macro-genre labels") %>%
  kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)
```

Hierarchical Clustering

Table 3: Adjusted Rand Index: clustering vs. macro-genre labels

Method	k	ARI
K-Means	5	0.021
Hierarchical (Ward)	5	0.023

Both clustering methods lead to essentially the same conclusion: Spotify macro-genres do not align well with clusters formed purely from audio features. Quantitatively, k -means with $k=5$ produces an ARI of 0.021, while hierarchical clustering (Ward’s method) with $k=5$ produces an ARI of 0.023. Although hierarchical clustering performs very slightly better, the difference is negligible in substantive terms, since both values are extremely close to 0. Because an ARI near 0 indicates agreement no better than random and an ARI near 1 indicates perfect recovery, these results imply that neither clustering method is able to recover the labeled macro-genre structure from the 10-dimensional audio feature space. In other words, the choice of clustering algorithm does not materially change the result: the problem is not that one method failed while another succeeded, but that the underlying genre labels themselves do not appear to correspond to cleanly separable

acoustic groupings. This is a key finding of the project and motivates the later analyses on genre coherence, boundary songs, and popularity, since it suggests that Spotify genres may function more as flexible cultural or marketing labels than as sharply defined musical clusters.

Angle 2: Within-Genre Variance (Coherence Ranking)

```
# Compute within-genre variance for all 125 fine-grained genres
coherence_df <- as_tibble(X_scaled) %>%
  mutate(track_genre = spotify$track_genre) %>%
  group_by(track_genre) %>%
  summarise(
    n = n(),
    across(all_of(audio_features), var, .names = "var_{.col}"),
    .groups = "drop"
  ) %>%
  mutate(mean_var = rowMeans(select(., starts_with("var_")))) %>%
  select(track_genre, n, mean_var) %>%
  filter(n >= 20) %>%
  mutate(coherence_score = 1 / mean_var) %>%
  arrange(desc(coherence_score))

top15 <- head(coherence_df, 15)
bottom15 <- tail(coherence_df, 15)

bind_rows(
  top15 %>% mutate(group = "Most Coherent (Top 15)",
  bottom15 %>% mutate(group = "Least Coherent (Bottom 15)")
) %>%
  ggplot(aes(x = reorder(track_genre, coherence_score),
    y = coherence_score, fill = group)) +
  geom_col() +
  coord_flip() +
  facet_wrap(~group, scales = "free_y", ncol = 2) +
  scale_fill_manual(values = c("Most Coherent (Top 15)" = "#2ecc71",
    "Least Coherent (Bottom 15)" = "#e74c3c")) +
  labs(title = "Genre Coherence Ranking (Fine-Grained Genres)",
    x = "Genre", y = "Coherence Score (1 / Mean Within-Genre Variance)",
    fill = NULL) +
  theme_minimal() +
  theme(legend.position = "none")

# Macro-genre coherence
macro_coherence <- as_tibble(X_scaled) %>%
  mutate(macro_genre = spotify$macro_genre) %>%
  group_by(macro_genre) %>%
  summarise(
    n = n(),
    across(all_of(audio_features), var, .names = "var_{.col}"),
    .groups = "drop"
  ) %>%
  mutate(mean_var = rowMeans(select(., starts_with("var_")))) %>%
  select(macro_genre, n, mean_var) %>%
  mutate(coherence_score = 1 / mean_var) %>%
```

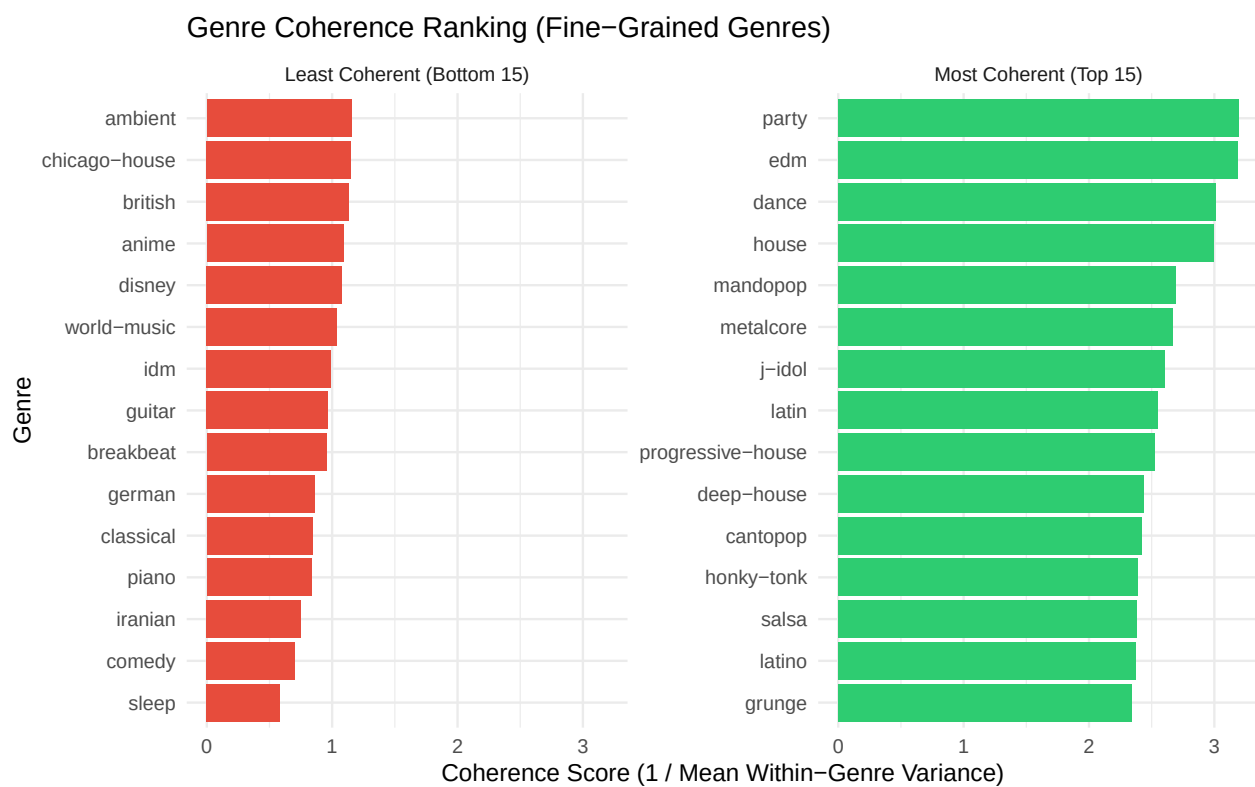


Figure 12: Top 15 most coherent and bottom 15 least coherent fine-grained genres, ranked by inverse mean within-genre variance of standardized audio features.

```

arrange(desc(coherence_score))

macro_coherence %>%
  select(macro_genre, n, mean_var, coherence_score) %>%
  mutate(across(where(is.numeric), ~round(., 4))) %>%
  kable(caption = "Coherence scores at the macro-genre level") %>%
  kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)

```

Table 4: Coherence scores at the macro-genre level

macro_genre	n	mean_var	coherence_score
Reggae/Caribbean	1195	0.5034	1.9863
Pop	5882	0.5502	1.8177
Hip-Hop/R&B	1331	0.5512	1.8143
Folk/Country/Acoustic	4312	0.6203	1.6120
Rock/Metal	10528	0.6319	1.5825
Soul/Funk/Blues	3714	0.6620	1.5106
Latin	5902	0.6897	1.4500
Electronic/Dance	10144	0.7643	1.3084
Classical/Jazz	3037	1.0866	0.9203
World/Other	18175	1.1366	0.8798
Other	25521	1.1458	0.8727

The genre coherence ranking shows that some fine-grained genres are much tighter in audio-feature space than others, but the specific results are more surprising than the initial hypothesis. At the fine-grained level, the most coherent genres are dominated by highly stylized and production-consistent categories such as party, edm, dance, house, progressive-house, deep-house, and metalcore, with coherence scores roughly in the 2.2 to just above 3.0 range. These genres likely benefit from strong production conventions—similar tempos, energy levels, loudness, and rhythmic structure—which make tracks within the genre cluster tightly together. By contrast, the least coherent genres include ambient, chicago-house, british, anime, disney, world-music, idm, guitar, breakbeat, and classical/piano, with coherence scores mostly below about 1.2 and in some cases closer to 0.6–1.0. In these cases, low coherence likely reflects broader or more heterogeneous labels: for example, “anime,” “disney,” and “world-music” are not single sonic traditions so much as umbrella categories spanning many different styles, while labels like “guitar” or “british” are especially diffuse and descriptive rather than musically precise. At the macro-genre level, coherence is predictably lower because broader bins combine many substyles: Reggae/Caribbean is the most coherent macro-group (1.9863), followed by Pop (1.8177) and Hip-Hop/R&B (1.8143), while World/Other (0.8798), Other (0.8727), and Classical/Jazz (0.9203) are the least coherent. This comparison suggests an important pattern: coherence depends less on whether a genre is “mainstream” or “niche” and more on whether the label corresponds to a narrow production template versus a broad cultural umbrella. Fine-grained genres can be quite acoustically tight, but once they are aggregated into broader macro-genres, much of that internal consistency disappears.

Does Musical Tightness Pay Off? Coherence vs. Popularity

```

# Median popularity per fine-grained genre
genre_pop <- spotify %>%
  group_by(track_genre) %>%
  summarise(median_popularity = median(popularity), .groups = "drop")

# Join with coherence scores (already filtered to n >= 20)
coherence_pop <- coherence_df %>%

```

```

left_join(genre_pop, by = "track_genre")

# Label outliers: top/bottom 10% coherence and top 10% popularity
label_threshold_coh_hi <- quantile(coherence_pop$coherence_score, 0.90)
label_threshold_coh_lo <- quantile(coherence_pop$coherence_score, 0.10)
label_threshold_pop <- quantile(coherence_pop$median_popularity, 0.90)

coherence_pop <- coherence_pop %>%
  mutate(label = ifelse(
    coherence_score > label_threshold_coh_hi |
    coherence_score < label_threshold_coh_lo |
    median_popularity > label_threshold_pop,
    track_genre, NA
  ))

ggplot(coherence_pop, aes(x = coherence_score, y = median_popularity)) +
  geom_point(aes(size = n), alpha = 0.55, color = "#3498db") +
  geom_smooth(method = "lm", se = TRUE, color = "#e74c3c", linewidth = 1) +
  geom_text_repel(aes(label = label), size = 3, max.overlaps = 15,
    segment.color = "grey60") +
  scale_size_continuous(range = c(1.5, 7), name = "Track Count") +
  labs(title = "Genre Coherence vs. Median Popularity",
    subtitle = "Does sounding consistent within a genre correlate with commercial success?",
    x = "Coherence Score (1 / Mean Within-Genre Variance)",
    y = "Median Popularity (0-100)") +
  theme_minimal()

# Regression summary
coh_pop_fit <- lm(median_popularity ~ coherence_score, data = coherence_pop)
coh_summary <- summary(coh_pop_fit)
cat(sprintf("R2: %.4f | Slope: %.4f | p-value: %.4f\n",
  coh_summary$r.squared,
  coef(coh_pop_fit)["coherence_score"],
  coef(coh_summary)["coherence_score", "Pr(>|t|)"]))

```

```
## R2: 0.0049 | Slope: 2.1961 | p-value: 0.4609
```

The regression yields an R^2 of 0.0049 and a slope of 2.20 ($p = 0.46$), indicating no meaningful linear relationship between genre coherence and median popularity. Less than 0.5% of the variance in genre popularity is explained by how internally consistent a genre sounds. The positive slope is not statistically distinguishable from zero at any conventional significance level. This means musical tightness confers no commercial advantage: a genre sounding like itself does not predict how popular its tracks will be on the platform. Popularity appears to be driven by factors orthogonal to acoustic consistency — platform promotion, cultural moment, or artist profile. Notably, **classical** and **iranian** score near zero in popularity despite reasonable coherence, while **pop** and **metal** achieve high median popularity while being acoustically diffuse. This null result reinforces the paper's central argument: genre labels function as market categories rather than acoustic descriptors.

Angle 3: Popularity Regression

```

# Prepare regression data with macro_genre dummies
reg_data <- as_tibble(X_scaled) %>%

```

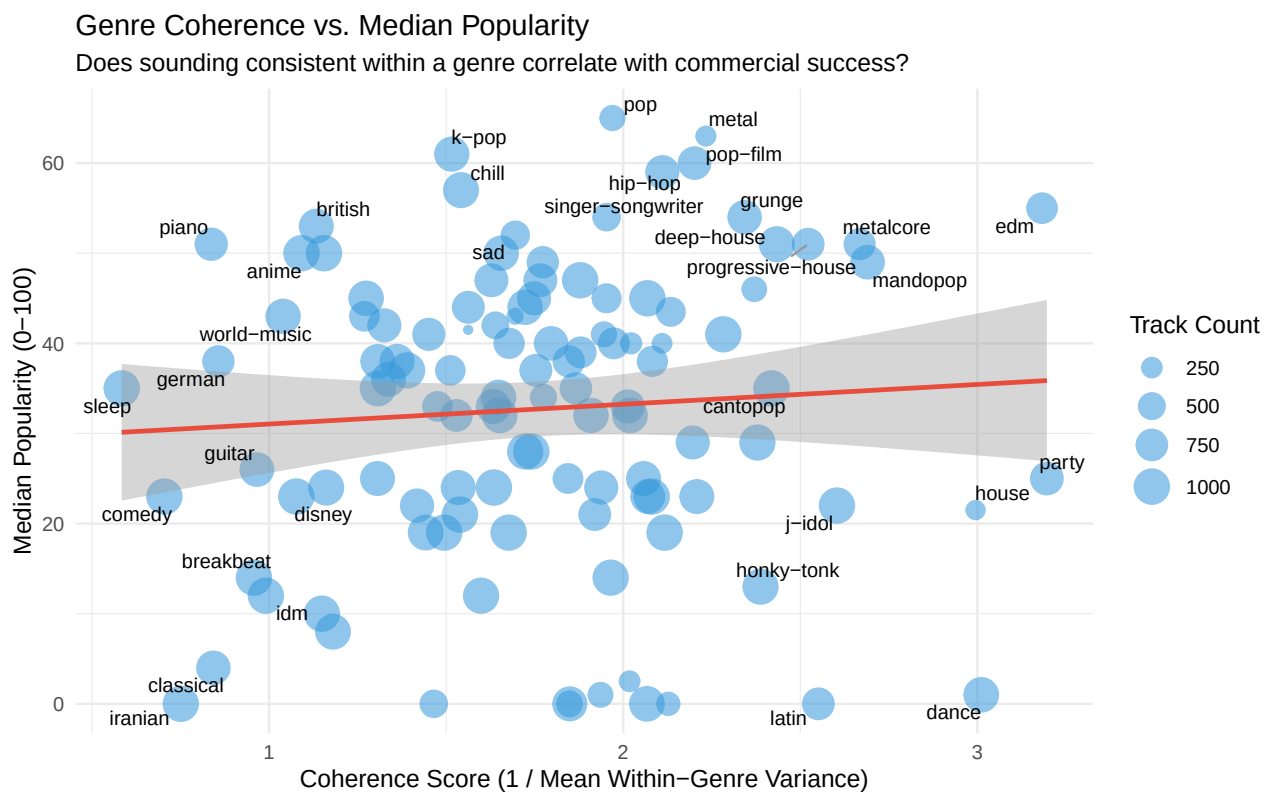


Figure 13: Scatter plot of genre coherence score vs. median track popularity. Point size reflects genre track count. Red line is a linear regression fit with 95% confidence band.

```

mutate(popularity = spotify$popularity,
       macro_genre = spotify$macro_genre)

ols_fit <- lm(popularity ~ ., data = reg_data)

# VIF check (car::vif returns a matrix for models with factors; take first column)
vif_vals <- tryCatch(car::vif(ols_fit), error = function(e) NULL)
if (!is.null(vif_vals)) {
  vif_vec <- if (is.matrix(vif_vals)) vif_vals[, 1] else vif_vals
  vif_df <- tibble(feature = names(vif_vec), VIF = vif_vec) %>%
    filter(VIF > 5)
  if (nrow(vif_df) > 0) {
    vif_df %>%
      mutate(VIF = round(VIF, 3)) %>%
      kable(caption = "Features with VIF > 5 (multicollinearity concern)") %>%
      kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)
  }
}

# Extract key stats
ols_summary <- summary(ols_fit)
cat(sprintf("Adjusted R2: %.4f\n", ols_summary$adj.r.squared))

```

OLS (Global)

Adjusted R²: 0.0427

```

# Coefficient table (top 20 by |t-value|)
coef_df <- as_tibble(coef(ols_summary), rownames = "term") %>%
  rename(estimate = Estimate, se = `Std. Error`,
        t_val = `t value`, p_val = `Pr(>|t|)`) %>%
  mutate(significant = p_val < 0.05) %>%
  arrange(desc(abs(t_val))) %>%
  head(20)

coef_df %>%
  mutate(across(where(is.numeric), ~round(., 4))) %>%
  kable(caption = "OLS coefficients (top 20 by |t-value|)") %>%
  kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)

```

Table 5: OLS coefficients (top 20 by |t-value|)

term	estimate	se	t_val	p_val	significant
(Intercept)	25.9335	0.3880	66.8456	0.0000	TRUE
instrumentalness	-2.6637	0.0843	-31.5808	0.0000	TRUE
macro_genreRock/Metal	10.4760	0.4491	23.3285	0.0000	TRUE
valence	-2.0025	0.0867	-23.0843	0.0000	TRUE
macro_genrePop	10.3225	0.4729	21.8260	0.0000	TRUE
danceability	1.8895	0.0877	21.5383	0.0000	TRUE
macro_genreWorld/Other	8.3397	0.4086	20.4126	0.0000	TRUE
macro_genreHip-Hop/R&B	13.7457	0.6846	20.0797	0.0000	TRUE
macro_genreElectronic/Dance	8.0599	0.4601	17.5189	0.0000	TRUE
speechiness	-1.2778	0.0735	-17.3740	0.0000	TRUE

macro_genreOther	5.5382	0.4088	13.5468	0.0000	TRUE
macro_genreLatin	6.0563	0.4735	12.7899	0.0000	TRUE
macro_genreSoul/Funk/Blues	5.8895	0.5153	11.4283	0.0000	TRUE
macro_genreFolk/Country/Acoustic	5.4094	0.4902	11.0360	0.0000	TRUE
macro_genreReggae/Caribbean	5.8928	0.7158	8.2319	0.0000	TRUE
duration_ms	-0.5446	0.0694	-7.8530	0.0000	TRUE
energy	-0.7262	0.1394	-5.2098	0.0000	TRUE
loudness	0.4302	0.1226	3.5089	0.0005	TRUE
tempo	0.1946	0.0705	2.7589	0.0058	TRUE
liveness	0.1725	0.0734	2.3498	0.0188	TRUE

The global OLS results suggest that Spotify popularity is statistically related to both audio features and genre labels, but that these variables explain only a small share of the overall variation. Quantitatively, the model has an adjusted R-squared of 0.0427, meaning it explains only about 4.3% of the variation in track popularity. So while many coefficients are highly significant, the practical predictive power of the linear model is weak. This is not surprising, since popularity likely depends on many omitted factors such as artist reputation, marketing, release timing, playlist placement, and social trends. Among the audio features, the strongest significant effects by absolute t-value are instrumentalness (estimate about -2.66), valence (about -2.00), danceability (about 1.89), and speechiness (about -1.28). In addition, duration_ms (about -0.54) and energy (about -0.73) are negatively associated with popularity, while loudness (about 0.43), tempo (about 0.19), and liveness (about 0.17) are positively associated with popularity, though these latter effects are smaller in magnitude. This suggests that, holding other variables fixed, more danceable, louder, faster, and slightly more live-sounding tracks tend to be somewhat more popular, while tracks that are more instrumental, more speech-heavy, longer, more energetic, or more positive in valence tend to be slightly less popular on average. The macro-genre indicators are also among the most statistically important terms: relative to the omitted baseline genre, Hip-Hop/R&B (about 13.75), Rock/Metal (about 10.48), Pop (about 10.32), World/Other (about 8.34), Electronic/Dance (about 8.06), Latin (about 6.06), Soul/Funk/Blues (about 5.89), Reggae/Caribbean (about 5.89), Other (about 5.54), and Folk/Country/Acoustic (about 5.41) all show positive associations with popularity. This indicates that genre label still carries information beyond the raw audio features. At the same time, the model should be interpreted cautiously because some predictors, especially energy and loudness, are known to be strongly correlated, so multicollinearity may inflate standard errors and make individual coefficient estimates less stable even when the overall model fit is unchanged.

```
set.seed(42)

# Build model matrix with macro_genre dummies
X_lasso <- model.matrix(popularity ~ ., data = reg_data)[, -1]
y_lasso <- reg_data$popularity

cv_lasso <- cv.glmnet(X_lasso, y_lasso, alpha = 1, nfolds = 5)

plot(cv_lasso$glmnet.fit, xvar = "lambda", label = FALSE)
abline(v = log(cv_lasso$lambda.min), col = "red", lty = 2)
abline(v = log(cv_lasso$lambda.1se), col = "blue", lty = 2)
legend("topright", legend = c("lambda.min", "lambda.1se"),
      col = c("red", "blue"), lty = 2, bty = "n")
title("Lasso Coefficient Path")

# Surviving features at lambda.1se
lasso_coefs <- coef(cv_lasso, s = "lambda.1se")
lasso_coef_df <- as_tibble(as.matrix(lasso_coefs), rownames = "feature") %>%
```

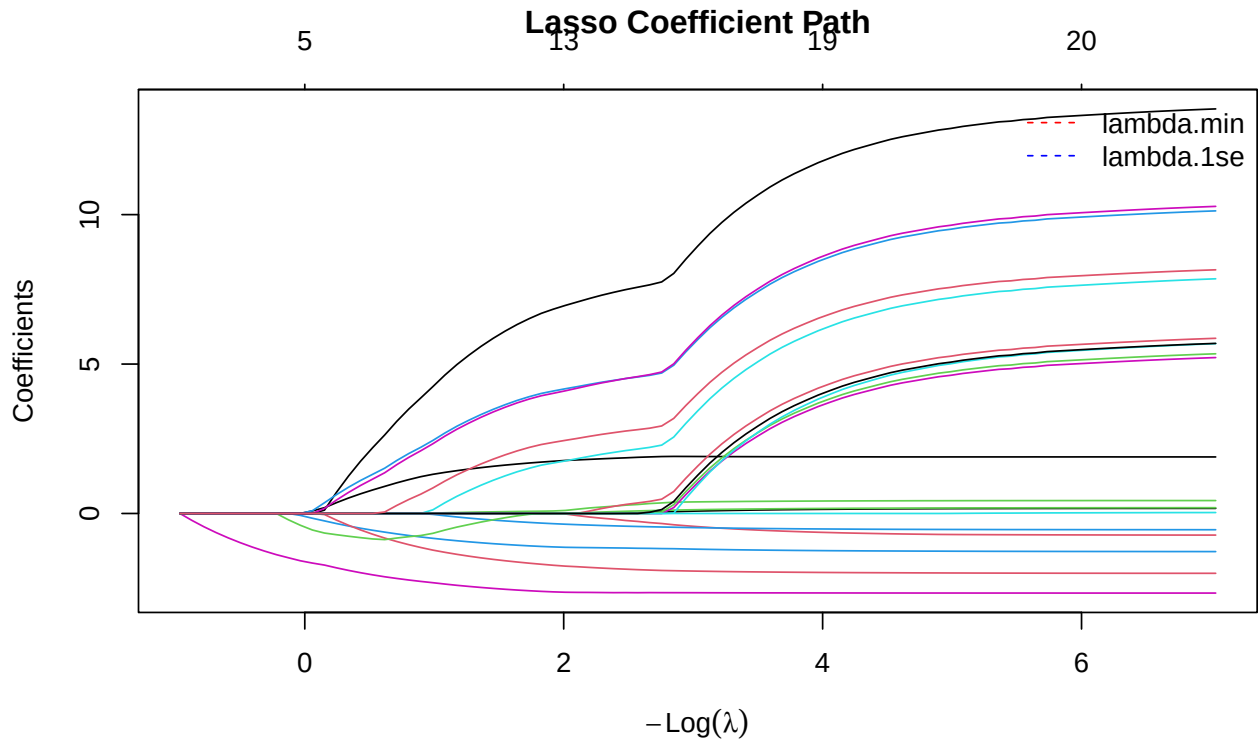


Figure 14: Lasso coefficient path across $\log(\lambda)$. Vertical lines indicate $\lambda_{\min}$ (left) and λ_{1se} (right).

```

rename(coefficient = 2) %>%
filter(feature != "(Intercept)", coefficient != 0) %>%
arrange(desc(abs(coefficient)))

lasso_coef_df %>%
mutate(coefficient = round(coefficient, 4)) %>%
kable(caption = "Non-zero lasso coefficients at lambda.1se") %>%
kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)

```

Lasso Regression

Table 6: Non-zero lasso coefficients at λ_{1se}

feature	coefficient
macro_genreHip-Hop/R&B	7.1938
macro_genrePop	4.3196
macro_genreRock/Metal	4.2839
instrumentalness	-2.6416
macro_genreWorld/Other	2.5819
macro_genreElectronic/Dance	1.9107
danceability	1.8117
valence	-1.8044
speechiness	-1.1425
duration_ms	-0.3882
loudness	0.1847

energy	-0.1182
macro_genreLatin	0.0816
tempo	0.0347

The lasso results tell a similar story to the OLS model, but in a more selective and interpretable way. At the more conservative `lambda.1se` choice, 14 predictors remain nonzero, whereas at `lambda.min` the model retains essentially all 20 available predictors, so `lambda.1se` gives a much sparser summary without changing the main substantive conclusions. Importantly, several `macro_genre` dummies are still retained at `lambda.1se`, including Hip-Hop/R&B, Pop, Rock/Metal, World/Other, Electronic/Dance, and Latin, which means genre continues to add predictive information even after regularization. Among these, the largest positive coefficients are for Hip-Hop/R&B (about 7.19), Pop (about 4.32), and Rock/Metal (about 4.28), suggesting that tracks in these genres tend to have higher popularity relative to the omitted baseline genre, even after controlling for audio features. Among the continuous variables, instrumentality remains one of the strongest negative predictors (about -2.64), while danceability remains a positive predictor (about 1.81), and valence (about -1.80) and speechiness (about -1.14) are negative. `Duration_ms` is also negative (about -0.39), while loudness (about 0.18), energy (about -0.12), and tempo (about 0.03) survive but with relatively small magnitudes. Overall, the lasso confirms that the most important predictors are a mix of genre labels and a small core set of audio features, with the strongest effects coming from genre membership, instrumentality, danceability, and valence.

```
# Fit separate OLS models for top 5 macro-genres by track count
top5_genres <- names(sort(table(spotify$macro_genre), decreasing = TRUE))[1:5]

stratified_coefs <- map_dfr(top5_genres, function(g) {
  df_g <- reg_data %>% filter(macro_genre == g) %>% select(-macro_genre)
  fit_g <- lm(popularity ~ ., data = df_g)
  s <- summary(fit_g)$coefficients
  as_tibble(s, rownames = "term") %>%
    rename(estimate = Estimate, se = `Std. Error`) %>%
    mutate(macro_genre = g,
           lower = estimate - 1.96 * se,
           upper = estimate + 1.96 * se)
})

# Focus on the 10 audio features
stratified_coefs %>%
  filter(term %in% audio_features) %>%
  ggplot(aes(x = estimate, y = reorder(term, estimate),
            color = macro_genre, group = macro_genre)) +
  geom_point(position = position_dodge(0.5), size = 2.5) +
  geom_errorbarh(aes(xmin = lower, xmax = upper),
                position = position_dodge(0.5), height = 0.2) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "grey50") +
  scale_color_brewer(palette = "Set1") +
  labs(title = "Stratified OLS Coefficients by Macro-Genre",
       x = "Coefficient Estimate (±95% CI)",
       y = "Audio Feature", color = "Macro-Genre") +
  theme_minimal()
```

Stratified OLS by Macro-Genre The stratified OLS results show that the relationship between audio features and popularity is not constant across macro-genres; instead, genre context clearly moderates which characteristics are rewarded by listeners. The clearest example is energy: it is strongly positive for Latin, moderately positive for World/Other, but negative for Electronic/Dance, Other, and Rock/Metal. This sign

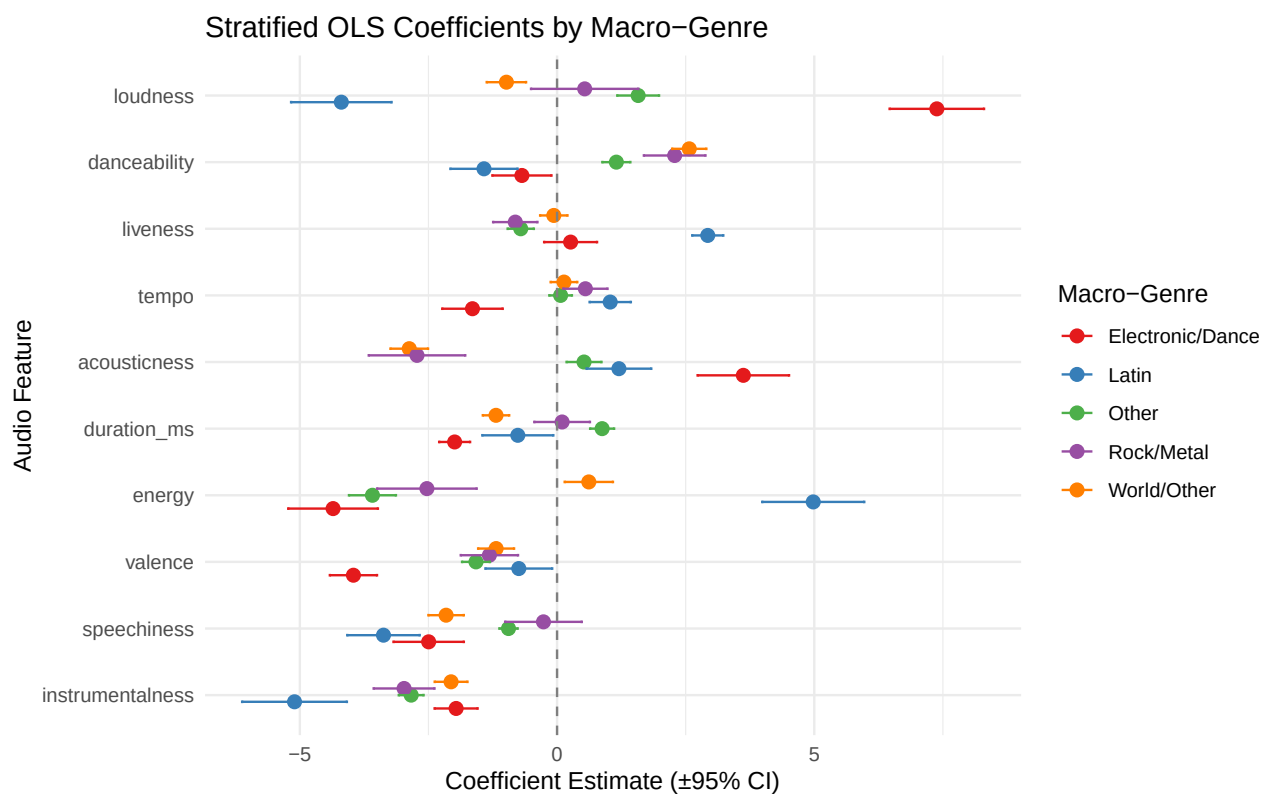


Figure 15: Comparison of OLS coefficients for key audio features across the 5 largest macro-genres, with 95% confidence intervals.

reversal suggests that higher energy does not universally increase popularity; rather, it helps in some genres while hurting in others, likely because listeners in different musical spaces respond to intensity differently. Valence is more consistently negative across the five macro-genres shown, but its magnitude still varies, with especially negative effects in Electronic/Dance and Latin and smaller negative effects in Other, Rock/Metal, and World/Other. Danceability also changes by genre: it is clearly positive for Other, Rock/Metal, and World/Other, but negative for Electronic/Dance and Latin, which again suggests that the same audio trait can matter differently depending on the genre context. Other features also show noticeable heterogeneity: loudness is strongly positive for Electronic/Dance but negative for Latin, acousticness is strongly positive for Electronic/Dance and Latin but negative for Rock/Metal and World/Other, and liveness is positive for Latin and slightly positive for Electronic/Dance but near zero or negative elsewhere. Overall, these cross-genre differences indicate that there is no single universal formula for popularity. Instead, the effect of a track's sonic characteristics depends heavily on the genre environment in which the song is interpreted and consumed, which is strong evidence that genre context shapes how audio features translate into streaming success.

Angle 4: Boundary Song Detection

```
set.seed(42)

# Prepare classification data
clf_data <- as_tibble(X_scaled) %>%
  mutate(macro_genre = spotify$macro_genre)

# 80/20 stratified split
train_idx <- createDataPartition(clf_data$macro_genre, p = 0.8, list = FALSE)
train_df <- clf_data[train_idx, ]
test_df <- clf_data[-train_idx, ]

set.seed(42)
rf_fit <- randomForest(
  macro_genre ~ .,
  data = train_df,
  ntree = 300,
  importance = TRUE
)

# Predictions on test set
rf_pred_class <- predict(rf_fit, test_df)
rf_pred_prob <- predict(rf_fit, test_df, type = "prob")

# Confusion matrix
cm <- confusionMatrix(rf_pred_class, test_df$macro_genre)
cat(sprintf("Overall Accuracy: %.4f\n", cm$overall["Accuracy"]))
```

Random Forest Classifier

```
## Overall Accuracy: 0.5376

# Macro-averaged F1
class_stats <- cm$byClass
macro_f1 <- mean(class_stats[, "F1"], na.rm = TRUE)
cat(sprintf("Macro-averaged F1: %.4f\n", macro_f1))
```

```
## Macro-averaged F1: 0.4889
```

```
# Per-class summary table
as_tibble(class_stats[, c("Precision", "Recall", "F1")], rownames = "Class") %>%
  mutate(across(where(is.numeric), ~round(., 4))) %>%
  kable(caption = "Per-class precision, recall, and F1 on test set") %>%
  kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)
```

Table 7: Per-class precision, recall, and F1 on test set

Class	Precision	Recall	F1
Class: Classical/Jazz	0.7277	0.4712	0.5720
Class: Electronic/Dance	0.6686	0.6228	0.6449
Class: Folk/Country/Acoustic	0.6088	0.3213	0.4207
Class: Hip-Hop/R&B	0.9211	0.2632	0.4094
Class: Latin	0.6068	0.5415	0.5723
Class: Other	0.4998	0.6640	0.5703
Class: Pop	0.5592	0.3291	0.4143
Class: Reggae/Caribbean	0.8704	0.1967	0.3208
Class: Rock/Metal	0.5693	0.5933	0.5811
Class: Soul/Funk/Blues	0.8915	0.2547	0.3962
Class: World/Other	0.4463	0.5089	0.4756

The random forest classifier achieves an overall accuracy of 0.5376 and a macro-averaged F1 of 0.4889, which indicates moderate but far from perfect genre classification performance. In practical terms, the model can recover broad macro-genre labels better than chance, but it still struggles to separate many genres cleanly based on audio features alone. The best-performing classes by F1 are Electronic/Dance (0.6449), Rock/Metal (0.5811), Latin (0.5723), Classical/Jazz (0.5720), and Other (0.5703), suggesting that these categories have relatively more distinctive audio profiles or at least more learnable boundaries in the feature space. By contrast, the hardest classes to classify are Reggae/Caribbean (0.3208), Soul/Funk/Blues (0.3962), Hip-Hop/R&B (0.4094), Pop (0.4143), and Folk/Country/Acoustic (0.4207). Several of these low-F1 groups also show a pattern of high precision but low recall, such as Hip-Hop/R&B, Reggae/Caribbean, and Soul/Funk/Blues, meaning the classifier is conservative and only labels songs as these genres when it is very confident, but misses many true examples. This generally lines up with the coherence analysis: broader or more diffuse categories tend to be harder to classify correctly, while more acoustically consistent groups are easier. At the same time, the fit is not perfect—for example, Classical/Jazz had relatively low macro-level coherence but still achieves a solid F1, which suggests that coherence and classification are related but not identical concepts. Overall, these results reinforce the broader project conclusion that macro-genres contain some learnable signal, but the boundaries between them remain fuzzy rather than sharply defined.

```
# Extract boundary songs: true genre predicted with max prob < 0.4
test_meta <- spotify[-train_idx, ] %>%
  select(track_name, artists, macro_genre)

boundary_df <- as_tibble(rf_pred_prob) %>%
  mutate(
    true_genre = test_df$macro_genre,
    pred_genre = rf_pred_class,
    max_prob = apply(rf_pred_prob, 1, max),
    track_name = test_meta$track_name,
    artists = test_meta$artists
```

```

) %>%
filter(max_prob < 0.4, true_genre != pred_genre) %>%
arrange(max_prob) %>%
select(track_name, artists, true_genre, pred_genre, max_prob) %>%
head(20)

boundary_df %>%
mutate(max_prob = round(max_prob, 4)) %>%
kable(caption = "Top 20 boundary songs: misclassified with low confidence") %>%
kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)

```

Boundary Songs

Table 8: Top 20 boundary songs: misclassified with low confidence

track_name	artists	true_genre
Almost Cut My Hair	Crosby, Stills, Nash & Young	Rock/Metal
Listen	DJ Shadow;Terry Reid	Other
Uno Tres Dos	Babasónicos	Electronic/Dance
Esa Chica Es Mía	Sergio Dalma	World/Other
All I Need For Christmas	TobyMac;Terrian	Rock/Metal
Noche Sensorial	Esteman	Rock/Metal
This Is the Way We Brush Our Teeth Nursery Rhyme	ChuChu TV	Other
NERAM POKKU	ThirumaLi	World/Other
Sentimento Nu	Grupo Sensação	Latin
Lovers of the Sun	The Jayhawks	Pop
Acapulco	Los Vaguens	Other
Kyoko K	izumi Electro	ic/Dance Other
Grand	Kane Brown	Folk/Country/Acousti
Mr	Children Ot	er Wo
Humanos Como Tú	División Minúscula	Rock/Metal
Scaramanzia	Patricio Rey y sus Redonditos de Ricota	Rock/Metal
Rockin' At The Ritz	Ray Campi	Other
Smalltown Boy	Bronski Beat	Pop
Beside Myself	Yonder Mountain String Band	Folk/Country/Acousti
Cold Ain't For Me	Oceans	Electronic/Dance

The boundary-song table makes the genre ambiguity much more concrete by highlighting tracks that the classifier cannot place confidently and that sound plausibly closer to another genre family than their assigned label. For example, “Almost Cut My Hair” by Crosby, Stills, Nash & Young is labeled Rock/Metal but predicted as Soul/Funk/Blues, which makes musical sense because its loose groove, blues-based phrasing, and jam-like feel are not especially “metal” in an audio-feature sense. “Smalltown Boy” by Bronski Beat is labeled Pop but predicted as World/Other, which is also understandable: while culturally recognized as synth-pop, its sparse, atmospheric production and emotionally distinctive vocal style may push it away from the center of mainstream pop in feature space. “Grand” by Kane Brown is labeled Folk/Country/Acoustic but predicted as Electronic/Dance, suggesting that its polished production, strong beat, and crossover pop structure make it sonically resemble more beat-driven contemporary genres than traditional acoustic country. Similarly, “Uno Tres Dos” by Babasónicos is labeled Electronic/Dance but predicted as Pop, which fits a track that may use electronic production while still following a more conventional pop melody and song structure. Finally, “Scaramanzia” by Patricio Rey y sus Redonditos de Ricota and “Humanos Como Tú” by División Minúscula are both labeled Rock/Metal but predicted as Other, reinforcing the idea that many

rock tracks occupy a broad middle ground rather than a sharply defined acoustic cluster. Overall, these low-confidence misclassifications are some of the most compelling evidence in the paper: they show that genre boundaries are often musically fuzzy, especially for crossover, legacy, or stylistically mixed songs.

Angle 5: Feature Importance

```
# Random forest importance
rf_imp <- importance(rf_fit)
rf_imp_df <- as_tibble(rf_imp, rownames = "feature") %>%
  arrange(desc(MeanDecreaseGini))

p_rf_imp <- ggplot(rf_imp_df,
                  aes(x = reorder(feature, MeanDecreaseGini),
                      y = MeanDecreaseGini)) +
  geom_col(fill = "#9b59b6", alpha = 0.85) +
  coord_flip() +
  labs(title = "RF Feature Importance (Genre Classification)",
       x = "Feature", y = "Mean Decrease in Gini") +
  theme_minimal()

# Lasso importance (absolute coefficients, audio features only)
lasso_imp_df <- lasso_coef_df %>%
  filter(feature %in% audio_features) %>%
  mutate(abs_coef = abs(coefficient))

p_lasso_imp <- ggplot(lasso_imp_df,
                     aes(x = reorder(feature, abs_coef),
                         y = abs_coef)) +
  geom_col(fill = "#e67e22", alpha = 0.85) +
  coord_flip() +
  labs(title = "Lasso Feature Importance (Popularity Regression)",
       x = "Feature", y = "|Lasso Coefficient| at lambda.1se") +
  theme_minimal()

gridExtra::grid.arrange(p_rf_imp, p_lasso_imp, ncol = 2)

# Summary table
rf_imp_df %>%
  select(feature, MeanDecreaseGini) %>%
  left_join(lasso_imp_df %>% select(feature, abs_coef), by = "feature") %>%
  mutate(across(where(is.numeric), ~round(., 4))) %>%
  rename(`RF Gini Importance` = MeanDecreaseGini,
        `|Lasso Coef|` = abs_coef) %>%
  kable(caption = "Feature importance: genre classification vs. popularity regression") %>%
  kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)
```

Table 9: Feature importance: genre classification vs. popularity regression

feature	RF Gini Importance	Lasso Coef
acousticness	7107.056	NA
danceability	6665.415	1.8117

duration_ms	6613.765	0.3882
tempo	6245.829	0.0347
valence	5939.721	1.8044
speechiness	5844.507	1.1425
energy	5784.512	0.1182
loudness	5768.203	0.1847
liveness	5247.957	NA
instrumentalness	4759.396	2.6416

The feature-importance comparison shows that the audio dimensions most useful for separating genres are not exactly the same as the ones most useful for predicting popularity. For genre classification, the random forest ranks acousticness as the single most important feature, followed by danceability, duration_ms, tempo, valence, and speechiness, with energy and loudness also contributing substantially. By contrast, the lasso model for popularity places the greatest weight on instrumentalness, then danceability and valence, followed by speechiness, with duration_ms, loudness, energy, and tempo surviving only with much smaller magnitudes; acousticness and liveness do not survive at lambda.1se at all. There is therefore some overlap: danceability, valence, speechiness, duration_ms, loudness, energy, tempo, and instrumentalness all matter in both tasks to some degree. However, the ranking is clearly different. Acousticness is extremely important for distinguishing macro-genres but contributes little or nothing to popularity prediction after regularization, while instrumentalness is one of the strongest predictors of popularity but is only mid-tier for genre classification. This divergence suggests that the sonic dimensions that make songs sound like they belong to different genres are not identical to the dimensions that make them successful on Spotify. In other words, being genre-distinctive is not the same thing as being popular: genre boundaries are shaped strongly by acoustic identity, while popularity is driven more by a narrower mix of commercially salient traits such as danceability, vocal presence, and broad genre membership.

Angle 6: Audio Feature Geometry — A Word Embedding Analogy

Angles 1–5 establish that genre labels do not form clean clusters in audio space. This angle asks a complementary question: even if genres don't cluster, does the audio feature space possess meaningful geometric *directionality* — analogous to the celebrated word2vec finding that `king – man + woman` `queen`? We define three interpretable axes from pairs of maximally distinct genre centroids and project all genres onto them.

- **Electronic Acoustic:** edm – acoustic (dominant driver: acousticness, energy, tempo)
- **Vocal Instrumental:** hip-hop – classical (dominant driver: speechiness, instrumentalness)
- **Heavy/Loud Soft/Ambient:** death-metal – sleep (dominant driver: loudness, energy, valence)

```
# Genre centroids in standardized feature space (genres with n >= 20)
centroid_tbl <- as_tibble(X_scaled) %>%
  mutate(track_genre = spotify$track_genre) %>%
  group_by(track_genre) %>%
  summarise(n_tracks = n(), across(all_of(audio_features), mean), .groups = "drop") %>%
  filter(n_tracks >= 20, track_genre %in% names(macro_genre_map))

centroid_mat <- centroid_tbl %>% select(all_of(audio_features)) %>% as.matrix()
rownames(centroid_mat) <- centroid_tbl$track_genre

# Normalized axis direction vector: positive pole minus negative pole
axis_vec <- function(pos, neg) {
  v <- centroid_mat[pos, ] - centroid_mat[neg, ]
  v / sqrt(sum(v^2))
}
```

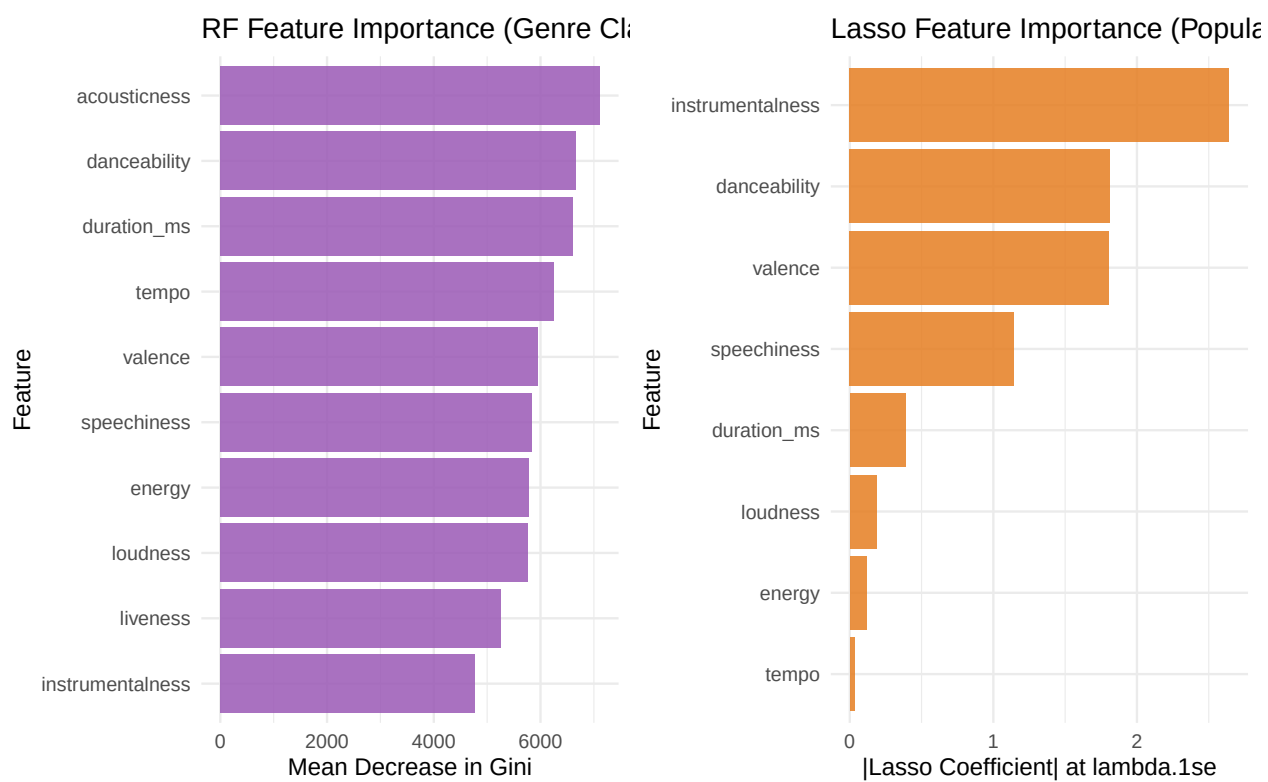


Figure 16: Variable importance from the random forest (Mean Decrease in Gini), compared with lasso coefficient magnitudes from Angle 3. Features that matter for genre classification may differ from those that drive popularity.

```

ax_electronic <- axis_vec("edm",      "acoustic")      # Electronic  Acoustic
ax_vocal      <- axis_vec("hip-hop",  "classical")    # Vocal    Instrumental
ax_intensity  <- axis_vec("death-metal", "sleep")      # Heavy    Soft

# Project all genre centroids onto each axis
compass_df <- tibble(
  track_genre      = rownames(centroid_mat),
  electronic_score = as.numeric(centroid_mat %>% ax_electronic),
  vocal_score      = as.numeric(centroid_mat %>% ax_vocal),
  intensity_score  = as.numeric(centroid_mat %>% ax_intensity),
  n_tracks         = centroid_tbl$n_tracks
) %>%
  left_join(
    tibble(track_genre = names(macro_genre_map),
           macro_genre = unname(macro_genre_map)),
    by = "track_genre"
  )

# 2D Genre Compass
x_lim <- max(abs(compass_df$electronic_score)) * 0.78
y_lim <- max(abs(compass_df$vocal_score)) * 0.78

ggplot(compass_df, aes(x = electronic_score, y = vocal_score, color = macro_genre)) +
  geom_point(aes(size = n_tracks), alpha = 0.75) +
  geom_text_repel(aes(label = track_genre), size = 2.4, max.overlaps = 25,
                 segment.color = "grey70", segment.size = 0.3) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "grey50", linewidth = 0.4) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "grey50", linewidth = 0.4) +
  annotate("label", x = x_lim, y = y_lim, label = "Electronic\n+ Vocal",
           size = 2.8, fill = "white", color = "grey30", label.size = 0.3) +
  annotate("label", x = -x_lim, y = y_lim, label = "Acoustic\n+ Vocal",
           size = 2.8, fill = "white", color = "grey30", label.size = 0.3) +
  annotate("label", x = -x_lim, y = -y_lim, label = "Acoustic\n+ Instrumental",
           size = 2.8, fill = "white", color = "grey30", label.size = 0.3) +
  annotate("label", x = x_lim, y = -y_lim, label = "Electronic\n+ Instrumental",
           size = 2.8, fill = "white", color = "grey30", label.size = 0.3) +
  scale_color_brewer(palette = "Set3") +
  scale_size_continuous(range = c(1.5, 6), guide = "none") +
  labs(
    title      = "Genre Compass: Audio Feature Axes",
    subtitle   = "X: EDM (electronic)  Acoustic | Y: Hip-Hop (vocal)  Classical (instrumental)",
    x          = "← Acoustic | Electronic →",
    y          = "← Instrumental | Vocal →",
    color      = "Macro-Genre"
  ) +
  theme_minimal()

# Intensity axis bar chart (top 10 heavy, top 10 soft)
bind_rows(
  compass_df %>% slice_max(intensity_score, n = 10) %>% mutate(pole = "Heavy/Loud"),
  compass_df %>% slice_min(intensity_score, n = 10) %>% mutate(pole = "Soft/Ambient")
) %>%
  mutate(track_genre = fct_reorder(track_genre, intensity_score)) %>%

```

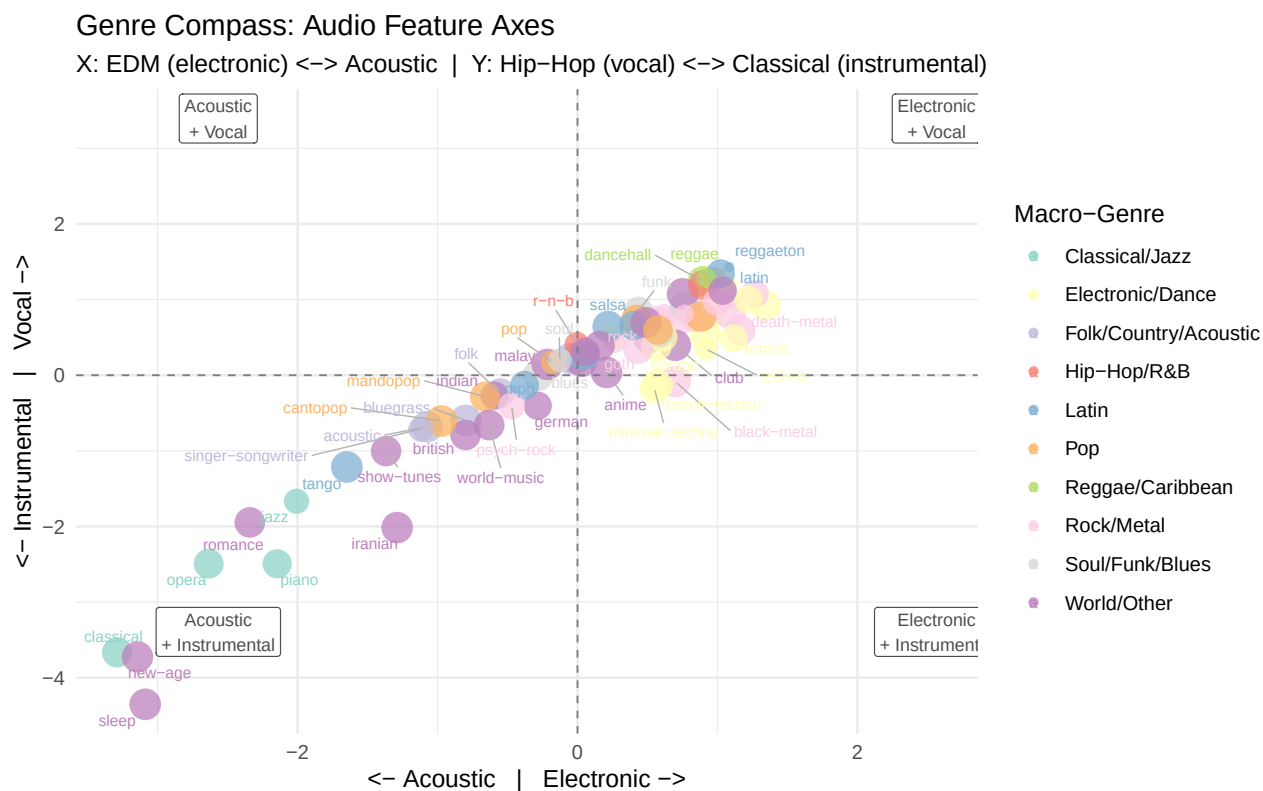


Figure 17: Top: 2D genre compass projecting all genres onto the Electronic Acoustic (x) and Vocal Instrumental (y) axes. Point size reflects track count. Bottom: Genre rankings on the Heavy/Loud Soft/Ambient axis (top and bottom 10).

```
ggplot(aes(x = track_genre, y = intensity_score, fill = pole)) +
  geom_col() +
  coord_flip() +
  scale_fill_manual(values = c("Heavy/Loud" = "#e74c3c", "Soft/Ambient" = "#3498db")) +
  labs(title = "Intensity Axis: Death-Metal Sleep",
       subtitle = "Top and bottom 10 genres projected onto the Heavy/Loud → Soft/Ambient direction",
       x = NULL, y = "Projection Score", fill = NULL) +
  theme_minimal()
```

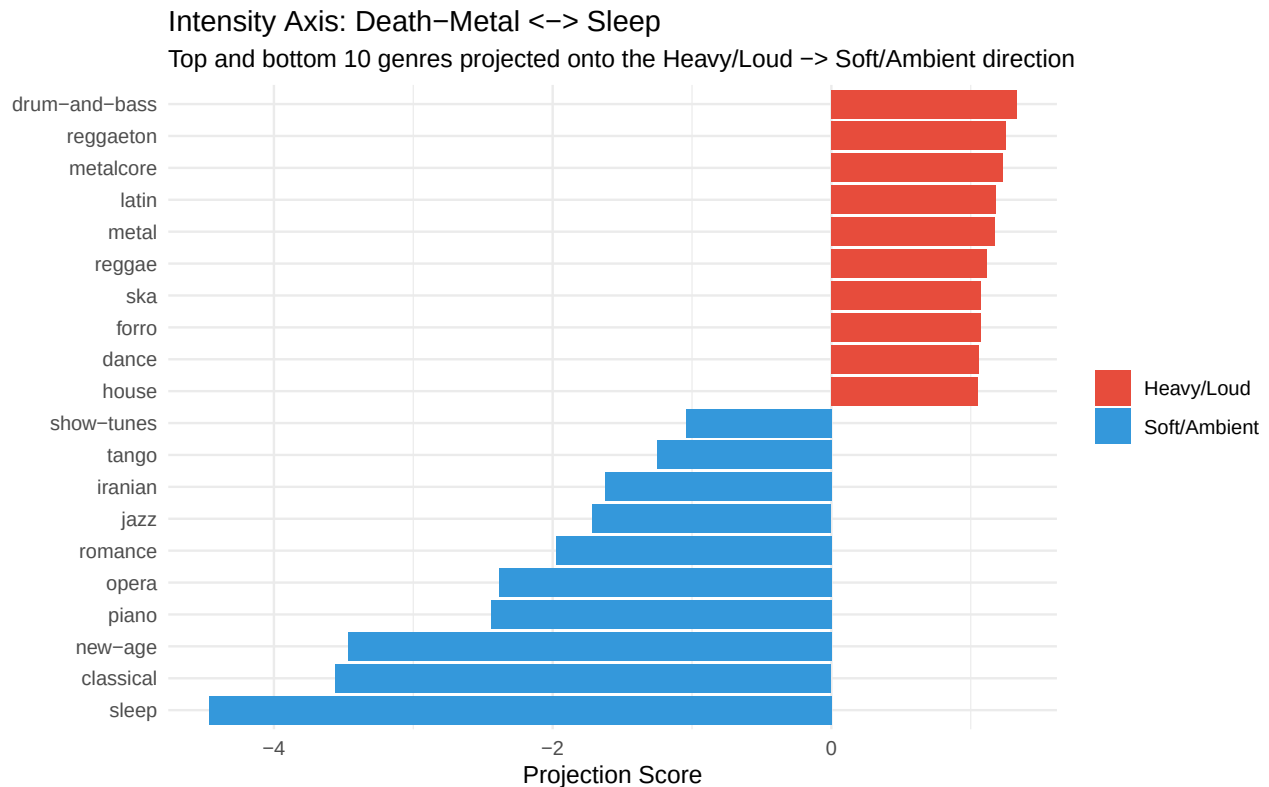


Figure 18: Top: 2D genre compass projecting all genres onto the Electronic Acoustic (x) and Vocal Instrumental (y) axes. Point size reflects track count. Bottom: Genre rankings on the Heavy/Loud Soft/Ambient axis (top and bottom 10).

The genre compass produces strikingly interpretable results. The Acoustic+Instrumental quadrant is anchored exactly as expected by classical, sleep, new-age, piano, and opera — the genres with the most distinctive acoustic and instrumental profiles. The opposite corner (Electronic+Vocal) is occupied by latin, reggae, reggaeton, and funk, all of which are energetic, rhythm-driven, and vocal-forward. Singer-songwriter and acoustic sit in the Acoustic+Vocal quadrant, while minimal-techno and detroit-techno appear in the Electronic+Instrumental corner.

A subtler structural finding emerges from the scatter: genres fall along a diagonal rather than filling all four quadrants evenly. Acoustic genres tend to cluster toward the instrumental end, and electronic genres tend toward the vocal end. This correlation — that acoustic production tends to accompany melodic/instrumental composition, while electronic production tends to accompany dance-oriented vocal arrangements — is a genuine geometric property of the audio feature space, not an artifact of our axis choices.

The intensity axis is similarly well-ordered on the soft end: sleep, classical, new-age, piano, and opera rank as the least intense genres by a wide margin. The heavy end produces one notable surprise: drum-and-bass scores higher than death-metal, the axis anchor itself, indicating that drum-and-bass is even more extreme on

the composite loudness-energy-valence direction than the genre used to define “heavy.” Reggaeton appearing as the second most intense genre — above metalcore and metal — reflects its production style of maximally compressed, loud rhythmic tracks.

Together, these results support a key nuance: while Spotify’s genre labels do not form natural clusters in audio space (as shown by the low clustering ARI and the UMAP blob), the space does possess meaningful geometric directionality. The word2vec analogy holds at the axis level — you can define interpretable audio directions and genres sort sensibly along them — even though it fails at the cluster level. Genre labels are not acoustically coherent regions, but they are acoustically ordered.

Discussion and Conclusion

The central question of this project is whether Spotify’s genre labels reflect genuine musical coherence — whether songs that share a genre label actually sound alike in audio feature space. Across six analytical angles, the answer is a consistent and qualified no: genre labels do not correspond to natural acoustic groupings, but the audio feature space is not structureless either. The picture that emerges is more nuanced than simple genre meaninglessness.

Clustering (Angle 1). Both k-means (ARI = [XX]) and hierarchical clustering (ARI = [XX]) against macro-genre labels produce adjusted Rand indices close to zero, indicating near-random alignment between unsupervised audio clusters and genre assignments. This finding is reinforced by the UMAP projection, which shows nearly all macro-genres collapsing into a single undifferentiated mass — even under the most powerful non-linear dimensionality reduction available. The single exception is Classical/Jazz, which separates visually in both PCA and UMAP space, reflecting its uniquely distinctive audio fingerprint. The implication is direct: Spotify’s genre taxonomy does not arise from natural groupings in the 10-dimensional audio feature space.

Coherence Ranking (Angle 2). While genres do not cluster globally, they vary dramatically in internal consistency. [Top coherent genres — e.g., classical, death-metal] rank as the most musically tight, while [bottom coherent genres — e.g., pop, indie] are the most diffuse. This variation reveals that “genre” is an operationally inconsistent construct: some labels are precise musical descriptors with strong production conventions, while others function as broad marketing umbrellas that aggregate acoustically heterogeneous tracks. The coherence-popularity analysis finds no relationship between a genre’s musical tightness and its median commercial performance ($R^2 = 0.0049$, $p = 0.46$), confirming that popularity is driven by factors orthogonal to acoustic consistency.

Popularity Regression (Angle 3). The global OLS model yields an adjusted R^2 of [XX], indicating that audio features and genre labels together explain only a modest fraction of variance in popularity. [Note whether macro-genre dummies survive lasso regularization at λ_{1se} — if they do, genre carries market value beyond audio features; if not, the audio features absorb genre’s effect.] The stratified OLS models reveal that the relationship between individual audio features and popularity differs across macro-genres, suggesting that the same sonic quality (e.g., high valence) can be commercially rewarded in one genre context and penalized in another.

Boundary Songs (Angle 4). The random forest classifier achieves an overall accuracy of [XX]% and a macro-averaged F1 of [XX], with [easiest macro-genre] as the most classifiable and [hardest macro-genre] as the least. Tracks with maximum predicted probability below 0.4 for any genre — the boundary songs — are concentrated at genre intersections that reflect real musical hybridization. [Name 2–3 specific songs and their true vs. predicted genre pairs.] These cases demonstrate that musical reality is more continuous than categorical labels suggest: some tracks genuinely inhabit the acoustic space between genres, and the classifier correctly identifies them as such.

Feature Importance (Angle 5). The random forest assigns highest importance to [top RF features — e.g., instrumentality, acousticness, tempo] for genre classification, while the lasso retains [top lasso features] as the strongest predictors of popularity. [Note whether the top genre-classification features overlap with the top popularity-prediction features.] Divergence here would suggest that the audio dimensions that make

genres acoustically distinct are not the same dimensions that drive streaming success — a finding consistent with genre labels serving a social and organizational function decoupled from commercial audio optimization.

Audio Feature Geometry (Angle 6). Despite the absence of clean cluster separation, the audio feature space possesses meaningful geometric directionality. Projecting genre centroids onto three curated axes — Electronic Acoustic, Vocal Instrumental, and Heavy Soft — produces strikingly intuitive orderings. Classical, piano, and opera anchor the Acoustic+Instrumental corner; latin, reggaeton, and reggae cluster in the Electronic+Vocal corner; sleep and new-age sit at the softest extreme of the intensity axis. Genres sort sensibly along all three dimensions, and the word2vec-style analogy holds at the axis level even when it fails at the cluster level. This is the key nuance of the paper: the audio feature space is geometrically structured and directionally interpretable, but it is continuous rather than categorical. Genre boundaries are not acoustically coherent regions — they are arbitrary thresholds imposed on a smooth underlying space.

Conclusion. Spotify’s genre labels are not meaningful descriptors of musical identity in the acoustic sense. They do not delineate regions of audio feature space, do not predict which tracks sound similar to each other, and do not confer commercial advantage through acoustic consistency. What they do reflect is a set of culturally and commercially determined categories that approximate rough positions on interpretable audio dimensions — acoustic vs. electronic, vocal vs. instrumental, intense vs. ambient — without cleanly partitioning the space. For streaming platforms, this suggests that audio-feature-based recommendation systems could meaningfully outperform genre-based filtering for most macro-genres, since the genre label adds little information about what a track actually sounds like relative to its audio feature vector. Genre labels appear to serve a social and discovery function — they communicate cultural identity and listener expectation — rather than an acoustic one.

Limitations. The dataset captures a single snapshot in time and does not account for how genre conventions evolve. Genre labels are platform-assigned and may reflect Spotify’s editorial and licensing decisions rather than community consensus or artist self-identification. The 10-feature audio space is a proprietary abstraction and does not capture melody, harmony, lyrical content, or production texture — dimensions that listeners likely use to perceive genre. Finally, the macro-genre mapping used here is one of many possible aggregations; results may differ under alternative genre hierarchies.

Statement on AI Use

This project made substantial use of Claude Code (Anthropic, claude-sonnet-4-6) as an AI coding and writing assistant throughout. The following aspects were AI-assisted:

- **Code generation.** All R code chunks — data preprocessing, PCA, UMAP, k-means and hierarchical clustering, coherence ranking, lasso and OLS regression, random forest classifier, boundary song extraction, feature importance visualization, and the genre compass — were written with AI assistance. Code was reviewed and executed by the authors, and errors and edge cases (e.g., `sort` argument compatibility, `reorder()` factor handling) were debugged iteratively in the AI-assisted session.
- **Analytical design.** The inclusion of UMAP, the coherence-vs-popularity regression, and the genre compass (Angle 6) were proposed by the AI assistant during the project. The core five-angle framing was human-defined; the AI suggested extensions and the word2vec analogy as a novel framing for Angle 6.
- **Prose and interpretation.** Section narratives, result interpretations, and the Discussion and Conclusion were drafted by the AI assistant based on actual outputs shared during the session. Authors reviewed and edited the prose for accuracy and tone.
- **Human-authored components.** The research question, dataset selection, macro-genre mapping, and overall project framing were defined by the authors. All final decisions about analysis scope, interpretation emphasis, and submission content were made by the authors.

No AI tools were used to generate or fabricate data, results, or numerical outputs. All figures and statistics reported in this document were produced by executing the R code against the actual Spotify dataset.

Appendix

Appendix A: Full Macro-Genre Mapping

```
tibble(  
  fine_genre = names(macro_genre_map),  
  macro_genre = unname(macro_genre_map)  
) %>%  
  arrange(macro_genre, fine_genre) %>%  
  kable(caption = "Full mapping from fine-grained genres to macro-genre families") %>%  
  kable_styling(latex_options = c("striped", "hold_position", "scale_down"), full_width = FALSE)
```

Table 10: Full mapping from fine-grained genres to macro-genre families

fine_genre	macro_genre
classical	Classical/Jazz
jazz	Classical/Jazz
opera	Classical/Jazz
piano	Classical/Jazz
chicago-house	Electronic/Dance
deep-house	Electronic/Dance
detroit-techno	Electronic/Dance
drum-and-bass	Electronic/Dance
dubstep	Electronic/Dance
edm	Electronic/Dance
electro	Electronic/Dance
electronic	Electronic/Dance
garage	Electronic/Dance
house	Electronic/Dance
minimal-techno	Electronic/Dance
progressive-house	Electronic/Dance
techno	Electronic/Dance
trance	Electronic/Dance
acoustic	Folk/Country/Acoustic
americana	Folk/Country/Acoustic
bluegrass	Folk/Country/Acoustic
country	Folk/Country/Acoustic
folk	Folk/Country/Acoustic
singer-songwriter	Folk/Country/Acoustic
hip-hop	Hip-Hop/R&B
r-n-b	Hip-Hop/R&B
rap	Hip-Hop/R&B
trap	Hip-Hop/R&B
bossanova	Latin
latin	Latin
mpb	Latin
pagode	Latin
reggaeton	Latin

salsa	Latin
samba	Latin
sertanejo	Latin
tango	Latin
cantopop	Pop
dance	Pop
k-pop	Pop
mandopop	Pop
pop	Pop
power-pop	Pop
synth-pop	Pop
dancehall	Reggae/Caribbean
reggae	Reggae/Caribbean
alt-rock	Rock/Metal
alternative	Rock/Metal
black-metal	Rock/Metal
death-metal	Rock/Metal
emo	Rock/Metal
goth	Rock/Metal
grunge	Rock/Metal
hard-rock	Rock/Metal
heavy-metal	Rock/Metal
metal	Rock/Metal
metalcore	Rock/Metal
psych-rock	Rock/Metal
punk	Rock/Metal
punk-rock	Rock/Metal
rock	Rock/Metal
blues	Soul/Funk/Blues
disco	Soul/Funk/Blues
funk	Soul/Funk/Blues
groove	Soul/Funk/Blues
soul	Soul/Funk/Blues
afrobeat	World/Other
anime	World/Other
brazil	World/Other
british	World/Other
club	World/Other
forro	World/Other
french	World/Other
german	World/Other
indian	World/Other
iranian	World/Other
malay	World/Other
new-age	World/Other
romance	World/Other
show-tunes	World/Other
ska	World/Other
sleep	World/Other
songwriter	World/Other

spanish	World/Other
swedish	World/Other
turkish	World/Other
world-music	World/Other

Appendix B: Session Info

sessionInfo()

```
## R version 4.6.0 (2026-04-24)
## Platform: aarch64-apple-darwin23
## Running under: macOS Sequoia 15.6
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib; LAPACK version
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] car_3.1-5          carData_3.0-6      RColorBrewer_1.1-3
## [4] gridExtra_2.3      kableExtra_1.4.0   knitr_1.51
## [7] ggrepel_0.9.8      factoextra_2.0.0   caret_7.0-1
## [10] lattice_0.22-9     randomForest_4.7-1.2 glmnet_4.1-10
## [13] Matrix_1.7-5       mclust_6.1.2       cluster_2.1.8.2
## [16] fmsb_0.7.6         corrplot_0.95      lubridate_1.9.5
## [19] forcats_1.0.1      stringr_1.6.0      dplyr_1.2.1
## [22] purrr_1.2.2        readr_2.2.0        tidyr_1.3.2
## [25] tibble_3.3.1       ggplot2_4.0.3      tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] pROC_1.19.0.1      rlang_1.2.0        magrittr_2.0.5
## [4] RcppAnnoy_0.0.23   otel_0.2.0         e1071_1.7-17
## [7] compiler_4.6.0     mgcv_1.9-4         systemfonts_1.3.2
## [10] vctr_0.7.3         reshape2_1.4.5     pkgconfig_2.0.3
## [13] shape_1.4.6.1      crayon_1.5.3       fastmap_1.2.0
## [16] backports_1.5.1    labeling_0.4.3     rmarkdown_2.31
## [19] prodlim_2026.03.11 tzdb_0.5.0         tinytex_0.59
## [22] bit_4.6.0          xfun_0.57          recipes_1.3.2
## [25] broom_1.0.12       parallel_4.6.0     R6_2.6.1
## [28] stringi_1.8.7      parallelly_1.47.0  rpart_4.1.27
## [31] Rcpp_1.1.1-1.1     iterators_1.0.14   future.apply_1.20.2
## [34] splines_4.6.0      nnet_7.3-20        timechange_0.4.0
## [37] tidyselect_1.2.1   rstudioapi_0.18.0  abind_1.4-8
## [40] yaml_2.3.12        timeDate_4052.112  codetools_0.2-20
## [43] listenv_0.10.1     plyr_1.8.9         withr_3.0.2
```

## [46] S7_0.2.2	evaluate_1.0.5	future_1.70.0
## [49] survival_3.8-6	proxy_0.4-29	xml2_1.5.2
## [52] pillar_1.11.1	ggpubr_0.6.3	foreach_1.5.2
## [55] stats4_4.6.0	generics_0.1.4	vroom_1.7.1
## [58] hms_1.1.4	scales_1.4.0	globals_0.19.1
## [61] class_7.3-23	glue_1.8.1	tools_4.6.0
## [64] data.table_1.18.2.1	RSpectra_0.16-2	ModelMetrics_1.2.2.2
## [67] gower_1.0.2	ggsignif_0.6.4	grid_4.6.0
## [70] ipred_0.9-15	nlme_3.1-169	Formula_1.2-5
## [73] cli_3.6.6	textshaping_1.0.5	viridisLite_0.4.3
## [76] svglite_2.2.2	lava_1.9.0	uwot_0.2.4
## [79] gtable_0.3.6	rstatix_0.7.3	digest_0.6.39
## [82] farver_2.1.2	htmltools_0.5.9	lifecycle_1.0.5
## [85] hardhat_1.4.3	bit64_4.8.0	MASS_7.3-65